

Macromedia University of Applied Sciences

Course Title: Applied Business Intelligence and Analytics

Name of Examiner: Meerah Karunanithi

To be completed by students:

306498

M-BUe-BUS-VT3-01-25WS_26460

Student ID number

Matriculation

Vanga

Syama Sanjay Reddy

Last name

First name

The student work will be submitted as:

(Please fill in the letter X in the appropriate box)

☒ Individual work

☐ Group work

Does only apply to group work: (Complete only if it is a group work)

If you are submitting a group work, please list the first and last names of all group members. The names must be entered electronically by the respective group members themselves. By entering the name, it is confirmed that the student agrees to submit the paper in the present form. Furthermore, by entering their names it is declared by the individual groups members to have created the project paper (in case of a group work: the part which the respective student has contributed to the paper and has marked accordingly within the paper) on their own, without the help of others. In the process, the student has not used any aids other than those cited in the listing of sources and literature. All passages taken either verbatim or in adapted form from publications are indicated as such. The work has not been submitted to another examination office in the same or a similar form. With the submission, the group members agree that all evaluations and notes of the examiners are deposited in the uploaded work. The group member who uploaded the work must make the correction notes available to the other group members.

1) _____
2) _____
3) _____
4) _____

5) _____
6) _____
7) _____
8) _____

Assessment of group work:

(Please fill in the letter X in the appropriate box)

☒ I apply for an individual evaluation (i.e. each member of the group will receive an individual mark)

☐ I apply for a group evaluation (i.e. each member of the group receives an identical grade)

Berlin, 23.01.2026

Vanga Syama Sanjay Reddy

Place/Date

Complete First Name and Last Name

Evaluation (according to grading scale), result of initial inspection: total points: _____

Date:

Name, first name First Examiner (to be filled in digitally)

To be completed by the examiner: (Text area for the second examiner)

Title Page for Digital Submission, Macromedia version 11/2022

LPATOP ANALYTICS STORE REPORT

By

SYAMA SANJAY REDDY VANGA-STUDENT ID 306498 MA Business Management

Table of Contents

1. Introduction	4
1.1 Business Context and Motivation	4
1.2 Business Problem Definition	4
1.3 Analytical Objectives	5
2. Data Sources, Scope, and Assumptions	5
2.1 Primary Data Source	5
2.2 Engineered and Derived Variables	6
2.3 Systems and Data Types	6
2.4 Business and Technical Scope	7
2.5 Key Assumptions	7
3. Data Cleaning and Feature Engineering	8
3.1 Initial Import and Exploration (Python & Excel)	8
3.2 Handling Missing and Invalid Values	8
3.3 Parsing and Standardizing Text Fields	8
3.4 Categorical Harmonization	9
3.5 Engineering Analytical Features	9
3.6 Final Validation and Export	10
4. Excel-Based Statistical Analysis and Dashboards	10
4.1 Descriptive Statistics of Laptop Prices	10
4.2 Hypothesis Test: Gaming vs Non-Gaming Prices	11
4.3 Pivot Tables and PivotCharts	12
4.4 Dynamic Excel Dashboard	12
5. Database Design and SQL Analysis	13
5.1 Logical Schema and Normalization	13
5.2 Implementation in MySQL (Analytical Table)	14
6. Power Pivot Data Model in Excel	16
7. Predictive Analytics in Python	17
7.1 Modelling Objective	17
7.2 Feature Selection and Target Variable	17
7.3 Model Choice: Random Forest Regression	18
7.4 Training and Evaluation Procedure	18
7.6 Visual Diagnostics (Static Python Dashboard)	19
8. Integrated Insights and Discussion	20
8.1 Summary of Key Findings	20
8.2 Business Implications	21
9. Future AI and Analytics Extensions	22
10. Limitations and Recommendations	23
10.1 Limitations	23
10.2 Recommendations	23
11. Conclusion	23

1. Introduction

1.1 Business Context and Motivation

The laptop market is highly fragmented, with hundreds of models differing in brand, technical specifications, price, and intended use case (e.g., gaming, study, work). Students and young professionals are often price sensitive but still need laptops with sufficient performance for study, productivity, and occasional gaming.

Traditional online laptop stores present long product lists with limited analytical support. Customers must manually compare specifications and prices across many models, and managers must rely on generic pricing rules instead of data-driven insights.

This project proposes a Laptop Analytics Store: a niche online laptop business that uses data analytics to:

- Help customers identify the best value-for-money laptops based on price, technical specifications and gaming performance.
- Support management decisions on pricing, product mix, and segment targeting using consistent analytical data foundation.
- The project uses an integrated toolset:
 - Excel for descriptive statistics and dynamic dashboards.
 - SQL for structured querying on a normalized laptop database.
 - Python for advanced data cleaning and Random Forest price prediction.

All three tools operate on a shared, cleaned dataset to ensure consistency across analyses.

1.2 Business Problem Definition

From a business perspective, the laptop store needs to answer questions such as:

- How are prices distributed across the portfolio, and how extreme is the price dispersion?
- Do gaming laptops command a significant price premium over non-gaming laptops?
- Which brands and specification combinations (RAM, SSD, screen size, specscore) are positioned as budget vs mid-range vs premium?
- Which laptops provide the highest technical performance per unit of price (value for money)?
- Can we predict the price of a laptop based on a small set of key features?

Addressing these questions enables:

- Better assortment planning (which models to stock).
- Data-driven pricing strategies (e.g., price bands per spec band).
- More relevant customer recommendations (e.g., best value gaming laptop under a budget).

1.3 Analytical Objectives

The project's analytical objectives are:

1. Describe the price structure of the laptop market using descriptive statistics and dashboards.
2. Compare gaming versus non-gaming laptops to determine if there is a statistically significant price difference.
3. Explain how technical features (RAM, SSD, screen size, specscore, release year) relate to laptop prices and brand positioning.
4. Design a normalized relational schema and a Power Pivot data model to support flexible SQL and Excel-based analysis.
5. Predict laptop prices using a Random Forest regression model in Python and evaluate the model's accuracy.
6. Integrate insights from Excel, SQL and Python into a coherent decision support story for both customers and management.

2. Data Sources, Scope, and Assumptions

2.1 Primary Data Source

- Kaggle "Laptop Uncleaned Dataset"

An open-source dataset containing laptop listings scraped from online marketplaces.

Key attributes in the raw CSV include:

- Product name (long text)
- Short description and marketing text
- Price
- Rating (as a score out of 100)
- Storage description (e.g., "1 TB HDD + 256 GB SSD")
- Display description (size and resolution)
- Operating system text

- Processor text
- Brand embedded in name
- Other textual attributes (e.g., thinness, presence of webcam, ports)

This data set forms the single source of truth that is reused in Excel, SQL and Python.

2.2 Engineered and Derived Variables

During data cleaning and feature engineering (see Section 3), several structured fields were derived:

- RAMGB – Numeric RAM size in GB (e.g., 8, 16, 32).
- SSDGB – Numeric SSD capacity in GB (0 if no SSD).
- HDDGB – Numeric HDD capacity (if present).
- ScreenInch – Numeric screen size in inches (e.g., 14.0, 15.6, 17.3).
- specscore – A composite “specification score” summarizing performance-relevant attributes.
- GamingFlag – Categorical flag (Gaming / Non-Gaming) that identifies whether the laptop is marketed as a gaming laptop.
- ReleaseYear – Year of release inferred from the dataset (e.g., 2020–2025).
- Price-per-spec metrics (in working sheets):
 - Price per GB of RAM/SSD.
 - Performance per 1,000 units of currency ($\text{specscore} / \text{price} * 1000$).

These engineered variables are critical for statistical analysis, dashboards, and the price prediction model.

2.3 Systems and Data Types

- **Systems used**

Excel (statistics, dashboards, Power Pivot).

MySQL (relational database and SQL queries).

Python (Pandas, scikit-learn for data cleaning and modelling).

- **Data types**

Structured data: Cleaned laptop table in CSV/Excel and SQL (numeric and categorical fields such as price, brand, RAMGB, SSDGB, rating100, specscore, ReleaseYear, GamingFlag).

Semi-structured data: Original textual descriptions for names, specs, and marketing text used initially for parsing but not directly used in final analysis.

2.4 Business and Technical Scope

- The project simulates an online laptop store using open-source data instead of internal ERP/CRM systems.
- ERP/CRM/SCM systems are not directly connected; their role is conceptually replaced by the Kaggle dataset.
- The focus is on product data and pricing, not on customer transaction data.
- The final dataset includes over 1,600 laptop records, covering:
 - Multiple brands (Acer, Asus, Dell, HP, Lenovo, Apple, etc.)
 - A variety of RAM, SSD, screen size and OS combinations
 - Both gaming and non-gaming models
 - Release years mainly from 2020–2025.

2.5 Key Assumptions

1. Data Accuracy Assumption

The Kaggle dataset is assumed to represent accurately the prices and specifications at the time of collection. Minor errors or outdated prices are accepted as noise.

2. Single-Price Assumption

Each row represents a unique laptop configuration with a single price, ignoring price variation across sellers or discounts.

3. SpecScore Construction Assumption

The specscore is treated as an externally provided composite score (from the dataset or pre-defined rules). It is assumed to be monotonically related to performance.

4. GamingFlag Assumption

Laptops are classified as Gaming or Non-Gaming based on explicit text indicators (e.g., “Gaming”, “ROG”, “TUF”, “Legion”) in the name/specs. Borderline cases (e.g., powerful non-gaming laptops) are kept as Non-Gaming to avoid over-complication.

5. ReleaseYear Assumption

When explicit release year is not directly provided, it is inferred from context in the dataset and assumed to be a reasonable approximation.

6. Currency and Inflation Assumption

All prices are treated in a single currency and at roughly the same time. Inflation and currency conversion effects are ignored.

7. Independence of Observations

Each row is treated as an independent observation, even if minor variants of the same

model appear multiple times; this is acceptable for descriptive and predictive purposes.

3. Data Cleaning and Feature Engineering

Data cleaning was carried out both in Python (Pandas) and Excel, to transform the unstructured raw dataset into an analysis-ready table.

3.1 Initial Import and Exploration (Python & Excel)

1. Steps performed:
2. Import raw CSV to Python using Pandas.
3. Inspect column types (numeric vs object), unique values, and missing value counts.
4. Calculate basic summary statistics on price and ratings (min, max, mean, median) to identify extreme outliers or data issues.
5. Export an intermediate cleaned file to Excel for manual sanity checks.

3.2 Handling Missing and Invalid Values

- Critical fields (price, rating100, specscore) were checked for missing or zero values.
- Rows with:
 - Missing price, or
 - Non-numeric price, or
 - Rating100 clearly outside valid range (e.g., negative or > 100), or
 - Missing specscore was removed to avoid distorted analysis.

This cleaning reduced noise and ensured that subsequent statistics and model training were based on realistic entries.

3.3 Parsing and Standardizing Text Fields

Several original columns contained combined text that needed to be parsed:

1. **Storage field** → SSDGB and HDDGB
 - a. Strings like “1 TB HDD + 256 GB SSD” were split into numeric HDD and SSD capacities.
 - b. If only SSD was present (e.g., “512 GB SSD”), HDDGB was set to 0.
 - c. If only HDD was present, SSDGB was set to 0.
 - d. Units (“GB”, “TB”) were converted to standardized GB.
2. **RAM field** → RAMGB

- a. Strings like “8 GB DDR4 RAM” were parsed to extract the integer amount of RAM (8, 16, 32 etc.).
- b. RAM type (DDR4, LPDDR5, etc.) was not modelled explicitly but indirectly captured via **specscore**.

3. Display field → ScreenInch

- a. Text like “15.6 39.62 cm, 1920 x 1080 px, IPS LCD Display” contained multiple values.
- b. The leading numeric (e.g., 15.6) was extracted as **ScreenInch**.

4. Operating System field → OSType

- a. Values like “Windows 11”, “Windows 10”, “macOS Monterey”, “Google Chrome” were mapped into a shorter OS category:
 - i. Windows
 - ii. Other (macOS, Chrome OS, etc.)

5. Processor field → Processor Family

- a. Text such as “Intel Core i5 12th Gen 4.4 Ghz” or “AMD Ryzen 7 5800H” was simplified:
 - i. Family (i3, i5, i7, i9, Ryzen 3/5/7/9, M2/M3 etc.)
 - ii. Clock speed is used only indirectly via **specscore**.
 - iii. Brand name standardization
 - iv. Brand was extracted and standardized (e.g., “ASUS” → “Asus”, “APPLE” → “Apple”) to avoid duplicates.

All parsing steps were first implemented in Python, then validated in Excel.

3.4 Categorical Harmonization

- Categorical text fields such as brand, OS, and Slim were cleaned to remove minor spelling variants.
- **GamingFlag** was derived from the product name and model line; any laptop clearly marketed for gaming (e.g., “ROG Strix”, “TUF Gaming”, “Legion”) was flagged as Gaming, otherwise Non-Gaming.

3.5 Engineering Analytical Features

Using Python and Excel, the following engineered features were created:

- **RAMGB, SSDGB, ScreenInch** – as numeric fields for modelling and aggregation.
 - **GamingFlag** – a categorical variable with values Gaming or Non-Gaming.
 - **ReleaseYear** – the year of release or listing (e.g., 2020–2025).

- SpecBand – a specification performance band (e.g., 70–79, 80–89, 90–100) derived from specscore.
- **Price-per-spec metrics:**
 - $\text{scoreper1000} = \text{specscore} / \text{price} * 1000$, which is used to rank laptops on value for money.

3.6 Final Validation and Export

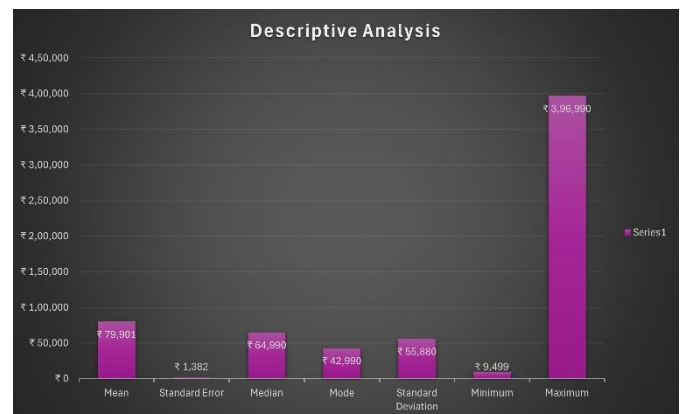
- Row counts before and after cleaning were compared.
- Random samples of rows were manually checked to ensure that numeric fields matched their corresponding text descriptions.
- The final cleaned dataset was exported as:
 - Excel tables for descriptive analysis and dashboards,
 - MySQL tables for SQL analysis,
 - Python DataFrame for modelling.

4. Excel-Based Statistical Analysis and Dashboards

4.1 Descriptive Statistics of Laptop Prices

Using Excel's Descriptive Statistics tool on the cleaned price column:

- Mean price: approximately 79,901
- Median price: 64,990
- Mode (most frequent price): 42,990
- Minimum price: 9,499
- Maximum price: 396,990
- Standard deviation: approximately 55,880
- Sample variance: around 3,122,524,299
- Range: about 387,491
- 95% confidence interval half-width: approx. 2,710



Interpretation and implications

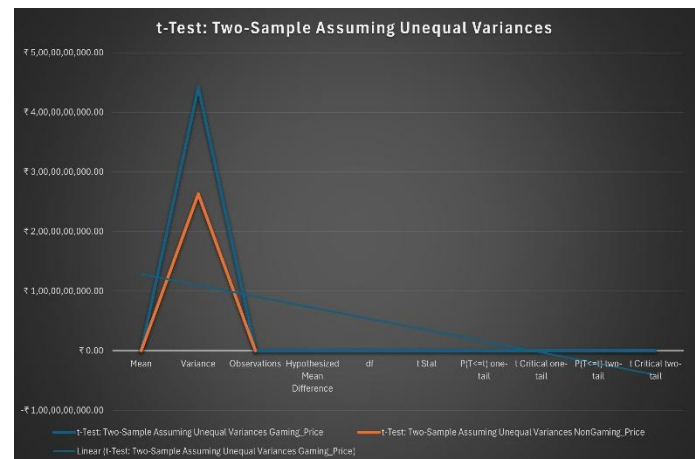
- The mean > median indicates a right-skewed distribution: most laptops are in the lower to mid-price range, but a small number of very high-end models pulls the mean upwards.

- The extremely large range and standard deviation confirm that price dispersion is very high; the store must handle both budget models and premium flagships.
- For management, the 95% confidence interval around the mean price provides an approximate range for “typical” prices, but given skewness and outliers, medians and quantiles may be more robust for decision-making.

4.2 Hypothesis Test: Gaming vs Non-Gaming Prices

A two-sample t-test assuming unequal variances was carried out in Excel, comparing:

- Gaming laptops (GamingFlag = “Gaming”)
- Non-gaming laptops (GamingFlag = “Non-Gaming”)



Key statistics from the t-test (sample data):

- Mean GamingPrice: $\approx 108,954$
- Mean NonGamingPrice: $\approx 73,690$
- p-value (two-tailed): extremely small (on the order of 10^{-16})
- t-statistic: approximately 8.48
- Degrees of freedom: about 363
- **Decision:** Reject the null hypothesis that the two means are equal.

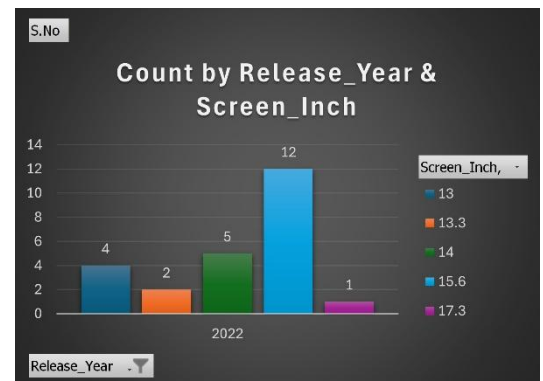
Conclusion

- Gaming laptops are statistically significantly more expensive than non-gaming laptops in this dataset.
- The price premium is consistent with higher specscore, more powerful GPUs and CPUs, and gamer-oriented designs.
- For the store, this justifies separate pricing and promotional strategies for gaming vs non-gaming segments.

4.3 Pivot Tables and PivotCharts

Several PivotTables were built on the cleaned laptop table in Excel:

1. Average price by brand
2. Shows how each brand is positioned on price:
 - a. Premium brands (e.g., Apple, some high-end Asus and Lenovo lines) have higher average prices.
 - b. Value brands (e.g., Acer, Infinix, some Asus VivoBook lines) cluster at lower price levels.
3. Average specscore by brand
 - a. Identifies performance-oriented brands, e.g., brands with gaming or high-performance series tend to have higher mean specscore.
4. Laptop count by ReleaseYear
 - a. Confirms that most laptops in the dataset are recent (2020–2025), indicating up-to-date products.
5. Average specscore by GamingFlag
 - a. Demonstrates that Gaming laptops have higher average specscore than Non-Gaming, reinforcing the t-test findings.



These PivotTables are visualized using PivotCharts (clustered column and bar charts) to make comparisons more intuitive.

4.4 Dynamic Excel Dashboard

A dynamic dashboard was created with:

Charts for:

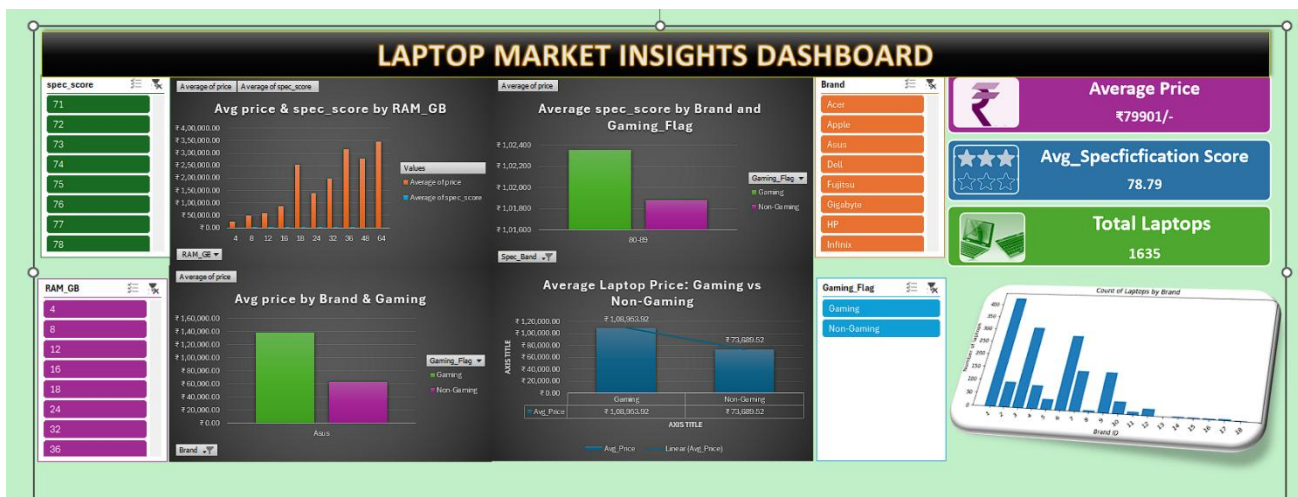
- Average price by brand.
- Number of laptops by release year.
- Average specscore by GamingFlag.
- Selected other views such as price vs specscore by RAM size.
 - Slicers for:
 - Brand

- RAMGB
- GamingFlag
- ReleaseYear

Usage

- Users can filter the charts interactively to:
 - Focus on specific brands (e.g., “Asus” or “Lenovo”).
 - Compare gaming vs non-gaming within a brand.
 - Observe how price and specscore evolve across release years.

This dynamic dashboard forms a front-end descriptive decision support for managers and provides quick insights during discussions.



5. Database Design and SQL Analysis

5.1 Logical Schema and Normalization

Initially, the dataset was a large unnormalized table with repeated brand, OS, and processor details in every row.

Normalization steps (conceptual side of the project):

1. 1NF (First Normal Form)

- Ensured all attributes are atomic (no comma-separated multi-valued fields).
- Defined a primary key LaptopID for each laptop row.

2. 2NF (Second Normal Form)

- Removed partial dependencies by separating data into:
 - BRAND (BrandID, BrandName, BrandCountry)
 - OPERATINGSYSTEM (OSID, OSName, Developer)

- iii. LAPTOP (LaptopID, LaptopName, Price, BrandID, OSID, RAMGB, SSDGB, Screen, Rating, GamingFlag, ReleaseYear, SpecScore)

3. 3NF (Third Normal Form)

- a. Created a separate PROCESSOR table to remove transitive dependencies:
 - i. PROCESSOR (ProcessorID, ProcessorName, Type, Cores)
- b. Linked LAPTOP to PROCESSOR via ProcessorID.

Benefits of normalization

- Reduced redundancy (brand and OS details stored once).
- Improved update consistency (changing OS name in one place).
- Provided clean master data to support future extensions (e.g., region-specific pricing, inventory).

5.2 Implementation in MySQL (Analytical Table)

For the analytical SQL work in this project, a single analytical table Laptops was created in a MySQL database laptopdss:

- Key fields:
 - laptopid (primary key)
 - name
 - brand
 - price
 - specscore
 - ramgb
 - releaseyear
 - screeninch
 - gamingflag

Representative SQL Queries and Objectives

SQL QUERIES				
Query No.	Query Title	SQL Query	Objective	Explanation
1	Full Laptop Dataset	SELECT * FROM Laptops;	Provide a complete view of all laptops available in the dataset.	Lists every record from the Laptops table, including name, brand, price, spec score, RAM, release year, screen size, and gaming flag. Serves as the base dataset for all further analysis.
2	Average Price by Brand	SELECT brand, AVG(price) AS avg_price FROM Laptops GROUP BY brand;	Understand how different brands are positioned in terms of average laptop price.	Groups data by brand and calculates average price, highlighting premium versus budget brands.
3	Average Spec Score by Brand	SELECT brand, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY brand;	Compare technical performance across brands.	Computes average spec score per brand to identify performance-oriented manufacturers.
4	Average Spec Score by Gaming Flag	SELECT gaming_flag, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY gaming_flag;	Test whether gaming laptops have better specifications.	Compares average spec scores between Gaming and Non-Gaming laptops to confirm performance differences.
5	Average Price and Spec Score by RAM Size	SELECT ram_gb, AVG(price) AS avg_price, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY ram_gb ORDER BY ram_gb;	Analyse the impact of RAM size on price and performance.	Shows how increasing RAM correlates with higher prices and better specifications.
6	Laptop Count by Release Year	SELECT release_year, COUNT(*) AS laptop_count FROM Laptops GROUP BY release_year ORDER BY release_year;	Assess how recent the laptop portfolio is.	Counts laptops by release year to evaluate dataset freshness and market relevance.
7	Brand by Gaming Flag Matrix	SELECT brand, gaming_flag, COUNT(*) AS laptop_count FROM Laptops GROUP BY brand, gaming_flag ORDER BY brand, gaming_flag;	Evaluate brand focus on gaming vs non-gaming segments.	Displays how each brand distributes its laptops between gaming and mainstream categories.

8	Cheapest Gaming Laptops	SELECT * FROM Laptops WHERE gaming_flag = 'Gaming' ORDER BY price LIMIT 5;	Identify budget-friendly gaming laptops.	Filters gaming laptops and lists the cheapest options, useful for cost-sensitive buyers.
9	Value-for-Money Ranking	SELECT name, brand, price, spec_score, ROUND(spec_score / (price / 1000), 2) AS score_per_1000 FROM Laptops ORDER BY score_per_1000 DESC;	Rank laptops by performance relative to price.	Uses a derived metric to highlight laptops offering the best performance per cost unit.
10	High-Spec Brands	SELECT brand, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY brand HAVING AVG(spec_score) > 80;	Identify consistently high-performance brands.	Filters brands whose average spec score exceeds a defined threshold, identifying premium brands.

5.3 SQL Dashboards

- Selected query results (e.g., average price by brand, gaming vs non-gaming specscore, value-for-money ranking) were exported to Excel.
- Static charts were built to create a SQL static dashboard in the presentation.
- These SQL-based views complement the Excel dynamic dashboard by providing transparent query logic behind the graphs.

6. Power Pivot Data Model in Excel

To support more complex slicing and dicing, a Power Pivot data model was built in Excel.

6.1 Star Schema Design

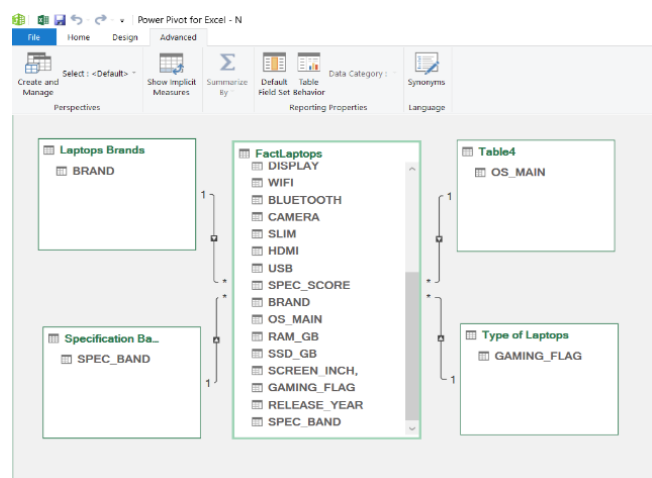
- **Fact Table: FactLaptops**

Contains one row per laptop with:

- Price, rating100, specscore
- RAMGB, SSDGB
- ScreenInch
- ReleaseYear
- GamingFlag
- Other technical fields

Dimension Tables:

- **DimBrand** – BrandID, BrandName



- DimOS (OSMain / OSType) – OSID, OSName, Developer
- DimSpecBand – SpecBandID, SpecBandLabel (e.g., “70–79”, “80–89”, “90–100”)
- DimType – Type of laptop (Gaming vs Non-Gaming or other categories)

6.2 Relationships

- Each dimension has a one-to-many relationships to FactLaptops:
 - DimBrand.BrandID → FactLaptops.BrandID
 - DimOS.OSID → FactLaptops.OSID
 - DimSpecBand.SpecBandID → FactLaptops.SpecBandID
 - DimType.TypeID → FactLaptops.TypeID

This structure:

- Simplifies the creation of PivotTables for different views.
- Allow the dashboard to filter by brand, OS, spec band and gaming type simultaneously.
- Mirrors the relational design from SQL but optimized for Excel analytics.

7. Predictive Analytics in Python

7.1 Modelling Objective

The goal is to predict laptop prices based on a small set of key structured features, providing:

- A “pricing assistant” that suggests price ranges for new or existing laptop configurations.
- A way to detect overpriced or underpriced models (by comparing actual vs predicted prices).

7.2 Feature Selection and Target Variable

- Target variable: Price
- Input features (X):
 - BrandID – encoded numeric brand identifier.
 - OSID – encoded operating system identifier.
 - specscore – composite performance score.
 - GamingFlag – encoded as binary (e.g., 1 for Gaming, 0 for Non-Gaming).
 - ReleaseYear – numeric.
 - RAMGB – numeric RAM size.

- SSDGB – numeric SSD capacity.
- ScreenInch – numeric screen size.

Categorical variables (Brand, OS, GamingFlag) were encoded appropriately (e.g., integer encoding or one-hot encoding, depending on implementation).

7.3 Model Choice: Random Forest Regression

A Random Forest Regressor was chosen because:

- It handles non-linear relationships and interactions between features.
- It is relatively robust to outliers and noise.
- It can manage mixed types of features comfortably.

Key model configuration (summary):

- Number of trees: 200
- Other hyperparameters: default or reasonably tuned settings in scikit-learn.

7.4 Training and Evaluation Procedure

1. Train–test split: The dataset was split into training and test sets (e.g., 80/20 split).
2. Model training: Random Forest fitted on the training set.
3. Prediction: Prices predicted on the test set.
4. Evaluation metrics:
 - Root Mean Squared Error (RMSE)
 - R^2 (coefficient of determination)
 - Mean Absolute Error (MAE)
 - Median Absolute Error
 - Mean Absolute Percentage Error (MAPE)

7.5 Model Performance

Reported performance on the test set:

- RMSE: $\approx 28,006$
- R^2 : ≈ 0.67
- The model explains about 67% of the variance in laptop prices.
- Average absolute error: $\approx 8,067$
- Median absolute error: $\approx 4,945$
- Average percentage error: $\approx 11\%$

Interpretation

- Most predicted prices are within a few thousand units of the true price, which is acceptable given the wide overall price range (from 9,499 to 396,990).
- An 11% average percentage error indicates that the model is reasonably accurate for price banding and relative pricing decisions, though not perfect at predicting exact list prices for every laptop.

7.6 Visual Diagnostics (Static Python Dashboard)

From the Python outputs, static visualizations were prepared (exported and used in the PPT):

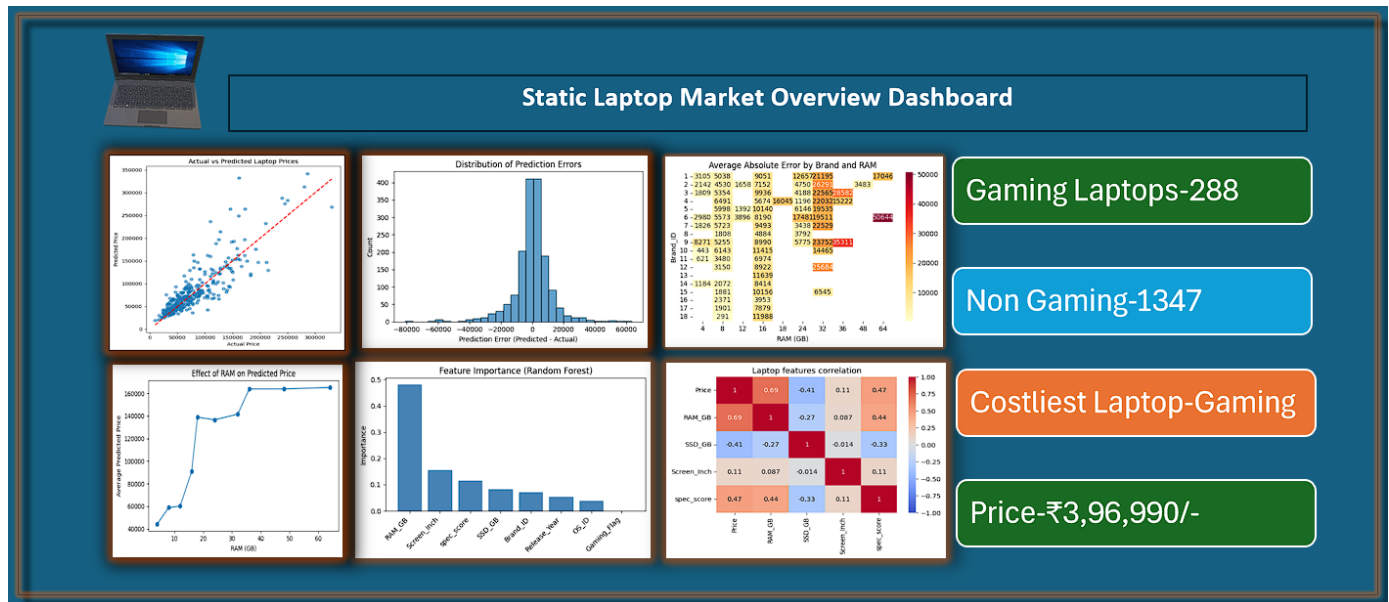
Actual vs Predicted Price Scatter Plot

- Points close to the diagonal line indicate good predictions.
- Outliers show laptops that are unusually cheap or expensive given their specs.

Error Distribution (Residual Histogram)

- Most errors are centered around zero with a spread roughly consistent with the computed RMSE and MAE.

These plots provide a visual check that the Random Forest is capturing the main structure of the pricespecification relationship.



8. Integrated Insights and Discussion

8.1 Summary of Key Findings

1. Price Distribution

- Laptop prices are highly dispersed and right-skewed.
- There is a large market of mid-range laptops centered around 60–80k, with a long tail of very high-end models.

2. Gaming vs Non-Gaming

- Gaming laptops are significantly more expensive than non-gaming laptops (confirmed by t-test).
- Gaming laptops also have higher average specscore, justifying the price premium.

3. Brand Positioning

- Some brands focus heavily on performance (high specscore), especially in gaming segments.
- Others specialise in budget or mid-range models with lower average price but still reasonable specscore.

4. Impact of Specifications

- Higher RAMGB and SSDGB are associated with higher prices and higher specscore.
- ScreenInch influences positioning (e.g., 17.3-inch gaming laptops typically at premium price points).

5. Value for Money

- The `scoreper1000` metric identifies laptops that deliver high performance per unit of price.
- These models are strong candidates for marketing as “Best Value Picks”.

6. Predictive Power

- The Random Forest model, using only a small set of features, achieves an R^2 of ~ 0.67 and $\sim 11\%$ average percentage error.
- This suggests that most of the price variation is explainable by brand, OS, specscore, gaming flag, release year, RAM, SSD and screen size.

8.2 Business Implications

For the simulated Laptop Analytics Store:

Pricing Strategy

- Use the Random Forest model as a pricing assistant to suggest reasonable price ranges for new or existing configurations.
- Identify laptops where actual price is far above predicted price as potential overpriced candidates (risk of low demand).
- Identify laptops where actual price is far below predicted price as potential underpriced bargains (opportunity to highlight as “Best Deals” or adjust prices upwards).

Assortment Planning

- **Maintain a clear mix of:**
 - Budget laptops for students (lower price, adequate specscore).
 - Mid-range laptops for professionals.
 - High-end gaming laptops for performance-oriented gamers.
 - Use SQL queries and dashboards to monitor how many models are offered in each segment and adjust the catalog accordingly.

Customer Decision Support

- Use the value-for-money metric to recommend laptops within a customer’s budget that maximize $\text{specscore} / \text{price}$.
- Integrate filters like Excel slicers on the website (brand, RAM, GamingFlag, release year).

Brand Strategy

- For brands with strong high-spec portfolios, position them as premium or performance brands.
- For brands with strong low-price, moderate specscore offerings, position them as value brands.

9. Future AI and Analytics Extensions

Building on the current project, several AI-driven agents can be envisioned:

Agent	Performance Measure	Environment	Actuators	Sensors
Laptop Recommendation Agent	Accuracy of recommendations, customer satisfaction, match with budget & usage, response time	Cleaned laptop dataset (Excel), customer preferences, recommendation interface	Display recommended laptops, rank options by value-for-money, filter by budget/specs	User inputs (budget, usage: student/gamer/professional), laptop specs (RAM_GB, SSD_GB, spec_score), price
Laptop Price Analysis Agent	Correct price classification, statistical accuracy, processing speed	Laptop price & specification dataset (Excel / SQL)	Generate price insights, classify laptops (budget / mid / premium), show trends	Laptop prices, brand, spec_score, gaming_flag, release_year
Laptop Specification Comparison Agent	Comparison accuracy, clarity, completeness of specs	Laptop dataset, selected laptops for comparison	Show comparison tables, highlight best specs, rank laptops	Laptop attributes (RAM_GB, SSD_GB, Screen_Inch, OS_Type, processor family)
Pricing Prediction Agent	Prediction accuracy (RMSE, R ²), error rate	Python Random Forest model, historical laptop data	Predict laptop prices, suggest optimal price range	Brand_ID, OS_ID, spec_score, RAM_GB, SSD_GB, Screen_Inch, gaming_flag
Gaming vs Non-Gaming Analysis Agent	Statistical validity, insight quality	Excel & SQL analytical environment	Generate t-test results, charts, insights	Gaming_Flag, laptop prices, spec_score
Dashboard Monitoring Agent	Timeliness of alerts, KPI accuracy	Excel Dynamic Dashboard	Refresh dashboards, highlight anomalies	PivotTables, slicer selections, KPI thresholds
Manager Insights Agent	Relevance of insights, response clarity	SQL database + Excel summaries	Answer business questions, generate reports	SQL query results, aggregated metrics
Customer Segmentation Agent (Future)	Clustering quality, segment interpretability	Python analytics environment	Assign customer segments, recommend offers	Purchase patterns, preferred specs, budget range

10. Limitations and Recommendations

10.1 Limitations

Data Source Limitation

- The dataset is derived from online listings and may not represent actual transaction prices or discount dynamics.

Feature Coverage

- Some important real-world factors (warranty, after-sales service, build quality perceptions) are not explicitly captured.

Model Limitations

- The Random Forest does not model time trends or market shocks; it provides a static mapping from specs to price.
- R^2 of 0.67 is good but not perfect; the model should be used for guidance rather than exact pricing.

10.2 Recommendations

- In a real deployment, integrate transactional, browsing and customer data to refine recommendations and price optimization.
- Periodically retrain the model as new laptop models enter the market and old ones are discontinued.
- Extend the data model to include promotions and discounts, enabling price elasticity analysis.
- Combine this analytics layer with a simple front-end web interface that exposes:
 - Interactive dashboards (like Excel slicers).
 - Recommendation widgets.
 - Price explanation (“Why is this laptop priced at X?” based on feature contributions).

11. Conclusion

This project demonstrates an end-to-end Business Analytics decision support system for a laptop store:

- A raw, unstructured Kaggle dataset was cleaned and transformed into a coherent analytical dataset using Python and Excel.
- Excel was used to compute detailed descriptive statistics and to build a dynamic dashboard with slicers for interactive exploration analysis.

- A normalized relational schema was designed, and a MySQL analytical table was populated to support SQL queries answering specific business questions about price, performance, brands and gaming segments.
- A Power Pivot star scheme in Excel enabled efficient multi-dimensional analysis of laptop data.
- A Random Forest regression model in Python successfully predicted laptop prices with an R^2 of around 0.67 and an average percentage error of ~11%, demonstrating that price is largely explainable from a small set of technical and categorical features.

Overall, the integrated use of Excel, SQL and Python created a robust, reusable analytical foundation for the Laptop Analytics Store. This foundation can be extended into AI-driven pricing, recommendation and monitoring agents, enabling a fully data-driven laptop retail business.



hochschule macromedia
university of applied sciences

Laptop Analytics Store Project

By Syama Sanjay Reddy Vanga
Syama Sanjay Reddy Vanga
Student Id 306498

Lecturer. Meerah Karunanithi

Business Idea – Laptop Analytics Store



- A niche online laptop store that helps students and working professionals choose the best-value laptops using data on price, specs, and gaming performance.

An online laptop store that uses data analytics to support pricing and product-mix decisions.
- Helps customers find the best value-for-money laptops based on price, technical specs (RAM, SSD, screen size, spec_score) and ratings.
- Enhances a typical laptop retail business by adding Excel statistics, dynamic dashboards, SQL analysis and predictive modelling in Python.
- Targets price-sensitive students, young professionals and performance-focused gamers.
- Built on a consolidated real-world laptop dataset with prices, brands, specs, ratings, gaming flag and release year.

Objective & Analytical Goals



OBJECTIVE:

To analyze laptop price patterns based on brand, specifications (RAM, SSD, screen size, spec_score) and release year.

ANALYTICAL GOALS:

1. To identify which technical features and gaming vs. non-gaming status have the strongest impact on price.
2. To use statistical analysis, Excel dashboards, SQL queries and Python models to generate data-driven insights for pricing and assortment decisions.
3. To support customers in choosing the best value-for-money laptops and support management in designing better product and pricing strategies.

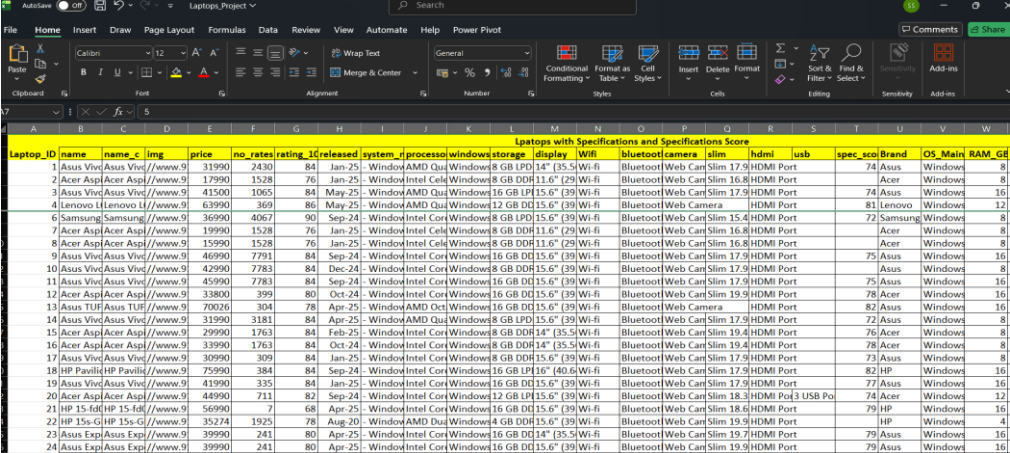
Research & Data Collection

- Conducted exploratory research on the laptop market focusing on prices, brands, technical specifications and customer ratings.
- Used a public “Laptop Uncleaned Dataset” from Kaggle as the primary data source (laptop models with prices, specs, ratings and descriptions from online listings).
- The original dataset contained raw attributes such as laptop name, detailed description, price, rating, storage, display information and other product features.
- Performed data cleaning and transformation in Excel and Python to remove inconsistencies, handle missing values and standardize fields (e.g., RAM, SSD, screen size).
- Engineered new analytical attributes such as RAM_GB, SSD_GB, Screen_Inch, spec_score, Gaming_Flag and Release_Year to support deeper business and pricing analysis.
- Prepared a final, structured dataset that is reused in all tools (Excel, SQL and Python) so the entire project is based on one consistent data foundation.

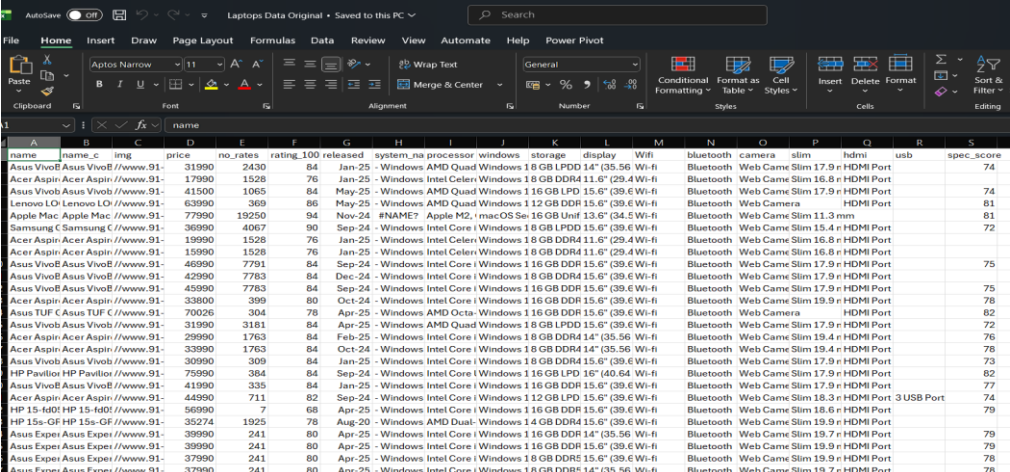


Data Cleaning & Feature Engineering (Excel + Python)

1. Imported the raw laptops dataset from CSV and inspected column types, unique values, and basic summary statistics.
2. Identified and removed rows with missing or invalid critical fields (price, rating_100, spec_score) to ensure reliable analysis.
3. Parsed complex text fields (e.g., storage, RAM, display) into separate numeric attributes such as RAM_GB, SSD_GB, HDD_GB and Screen_Inch.
4. Standardized operating system values (Windows 10, Windows 11, macOS, ChromeOS, DOS) into a consistent OS_Type variable.
5. Normalized processor information by extracting processor family (i3/i5/i7, Ryzen 3/5/7, M2/M3, etc.) and clock speed for easier comparison.
6. Cleaned and harmonized categorical text (brand names, model names, "Slim" flag, Wifi/Bluetooth indicators) to remove duplicates and typos.
7. Converted price and ratings columns to numeric format and verified realistic value ranges, flagging extreme outliers for review.
8. Removed irrelevant or unusable columns (e.g., long marketing text, partial URLs) and kept a focused subset of analysis-ready fields.
9. Created additional engineered variables (e.g., Price_per_GB, Price_per_Inch, Gaming_Flag) to better capture value and performance.
10. Performed final validation checks (row counts before/after cleaning, missing value audit) to document the impact of the cleaning steps.
11. Performed initial cleaning and feature extraction in Python (Pandas) to handle complex text parsing and type conversions efficiently.
12. Used Python to split storage strings into SSD_GB / HDD_GB, extract RAM_GB and Screen_Inch, and standardize OS and processor families.
13. Implemented Python checks for missing values, duplicates, and outliers (e.g., unrealistic prices/ratings) and filtered or corrected them.
14. Exported the cleaned, engineered dataset from Python back into Excel for further analysis, visualization, and dashboarding.



Laptop_ID	name	name_c	img	price	no_rates	rating_100	released	system_processor	processor	storage	display	Wifi	bluetooth	camera	slim	hdmi	usb	spec_score	brand	OS_Main	RAM_GB	HDD_GB
1	Asus VivoBook 15	Asus VivoBook 15		31990	2430	84	Jan-25	Windows	AMD Quad	Windows	8 GB LPD 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	74	Asus	Windows	8	
2	Acer Aspire 5	Acer Aspire 5		17990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer	Windows	8	
3	Asus VivoBook 15	Asus VivoBook 15		41500	1065	84	May-25	Windows	AMD Quad	Windows	16 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	74	Asus	Windows	16	
4	Lenovo L15	Lenovo L15		63990	369	86	May-25	Windows	AMD Quad	Windows	12 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam		HDMI Port	81	Lenovo	Windows	12	
5	Samsung Chromebook	Samsung Chromebook		36990	4067	90	Sep-24	Windows	Intel Core	Windows	8 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 15.4	HDMI Port	72	Samsung	Windows	8	
6	Asus VivoBook 15	Asus VivoBook 15		19990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer	Windows	8	
7	Acer Aspire 5	Acer Aspire 5		15990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer	Windows	8	
8	Asus VivoBook 15	Asus VivoBook 15		46990	7791	84	Sep-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus	Windows	16	
9	Asus VivoBook 15	Asus VivoBook 15		42990	7783	84	Dec-24	Windows	Intel Core	Windows	8 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus	Windows	8	
10	Asus VivoBook 15	Asus VivoBook 15		45990	7783	84	Sep-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus	Windows	16	
11	Asus VivoBook 15	Asus VivoBook 15		33800	399	80	Oct-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	78	Acer	Windows	16	
12	Asus VivoBook 15	Asus VivoBook 15		70026	304	78	Apr-25	Windows	AMD Quad	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam		HDMI Port	82	Asus	Windows	16	
13	Asus VivoBook 15	Asus VivoBook 15		31990	3181	84	Apr-25	Windows	AMD Quad	Windows	8 GB LPD 15.6"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	72	Asus	Windows	8	
14	Acer Aspire 5	Acer Aspire 5		29990	1763	84	Feb-25	Windows	Intel Core	Windows	8 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 19.4	HDMI Port	76	Acer	Windows	8	
15	Acer Aspire 5	Acer Aspire 5		33990	1763	84	Oct-24	Windows	Intel Core	Windows	8 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 19.4	HDMI Port	78	Acer	Windows	8	
16	Asus VivoBook 15	Asus VivoBook 15		30990	309	84	Jan-25	Windows	Intel Core	Windows	8 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	73	Asus	Windows	8	
17	Asus VivoBook 15	Asus VivoBook 15		75990	384	84	Sep-24	Windows	Intel Core	Windows	16 GB LPD 16"	(40.6	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	82	HP	Windows	16	
18	HP Pavillion HP	HP Pavillion HP		41990	335	84	Jan-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	77	Asus	Windows	16	
19	Acer Aspire 5	Acer Aspire 5		44990	711	82	Sep-24	Windows	Intel Core	Windows	12 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 18.3	HDMI Port	74	Acer	Windows	12	
20	HP 15-fd0 HP	HP 15-fd0 HP		56990	7	68	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	79	HP	Windows	16	
21	HP 15s-G HP	HP 15s-G HP		35274	1925	78	Aug-20	Windows	AMD Quad	Windows	4 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus	Windows	16	
22	Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	79	Asus	Windows	16	
23	Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus	Windows	16	
24	Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus	Windows	16	



name	name_c	img	price	no_rates	rating_100	released	system_processor	processor	storage	display	Wifi	bluetooth	camera	slim	hdmi	usb	spec_score	brand
Asus VivoBook 15	Asus VivoBook 15		31990	2430	84	Jan-25	Windows	AMD Quad	Windows	8 GB LPD 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	74	Asus
Acer Aspire 5	Acer Aspire 5		17990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer
Asus VivoBook 15	Asus VivoBook 15		41500	1065	84	May-25	Windows	AMD Quad	Windows	16 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	74	Asus
Lenovo L15	Lenovo L15		63990	369	86	May-25	Windows	AMD Quad	Windows	12 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam		HDMI Port	81	Lenovo
Samsung Chromebook	Samsung Chromebook		36990	4067	90	Sep-24	Windows	Intel Core	Windows	8 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 15.4	HDMI Port	72	Samsung
Asus VivoBook 15	Asus VivoBook 15		19990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer
Acer Aspire 5	Acer Aspire 5		15990	1528	76	Jan-25	Windows	Intel Celer	Windows	8 GB DDR 11.6"	(29	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	74	Acer
Asus VivoBook 15	Asus VivoBook 15		46990	7791	84	Sep-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus
Asus VivoBook 15	Asus VivoBook 15		42990	7783	84	Dec-24	Windows	Intel Core	Windows	8 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus
Asus VivoBook 15	Asus VivoBook 15		45990	7783	84	Sep-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	75	Asus
Acer Aspire 5	Acer Aspire 5		33800	399	80	Oct-24	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	78	Acer
Asus VivoBook 15	Asus VivoBook 15		70026	304	78	Apr-25	Windows	AMD Quad	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam		HDMI Port	82	Asus
Asus VivoBook 15	Asus VivoBook 15		31990	3181	84	Apr-25	Windows	AMD Quad	Windows	8 GB LPD 15.6"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	72	Asus
Acer Aspire 5	Acer Aspire 5		29990	1763	84	Feb-25	Windows	Intel Core	Windows	8 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 19.4	HDMI Port	76	Acer
Acer Aspire 5	Acer Aspire 5		33990	1763	84	Oct-24	Windows	Intel Core	Windows	8 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 19.4	HDMI Port	78	Acer
Asus VivoBook 15	Asus VivoBook 15		30990	309	84	Jan-25	Windows	Intel Core	Windows	8 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	73	Asus
HP Pavillion HP	HP Pavillion HP		75990	384	84	Sep-24	Windows	Intel Core	Windows	16 GB LPD 16"	(40.6	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	82	HP
HP 15-fd0 HP	HP 15-fd0 HP		41990	335	82	Sep-24	Windows	Intel Core	Windows	12 GB LPD 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 18.3	HDMI Port	74	Acer
HP 15s-G HP	HP 15s-G HP		56990	7	68	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 16.8	HDMI Port	79	HP
Asus VivoBook 15	Asus VivoBook 15		35274	1925	78	Aug-20	Windows	AMD Quad	Windows	4 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus
Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 14"	(35.5	Wi-Fi	Bluetooth	Web Cam	Slim 17.9	HDMI Port	79	Asus
Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus
Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus
Asus VivoBook 15	Asus VivoBook 15		39990	241	80	Apr-25	Windows	Intel Core	Windows	16 GB DDR 15.6"	(39	Wi-Fi	Bluetooth	Web Cam	Slim 19.9	HDMI Port	79	Asus

Data Sources & Data Types



Primary data source (Open-source dataset): Kaggle “Laptop Uncleaned Dataset” with laptop listings, prices, specifications and ratings from online marketplaces.

Secondary data sources:

Engineered fields created during cleaning (RAM_GB, SSD_GB, Screen_Inch, spec_score, Gaming_Flag, Release_Year).

Summary tables and pivot outputs generated in Excel and Python for further analysis.

Systems used (as requested in instructions):

ERP / CRM / SCM systems: Not directly used; business is simulated using open-source data instead of internal ERP/CRM.

Open-source datasets: Kaggle laptop dataset (core source for all analysis).

Websites / social media: Indirectly via the Kaggle dataset, which aggregates product information from online listings.

Type of data:

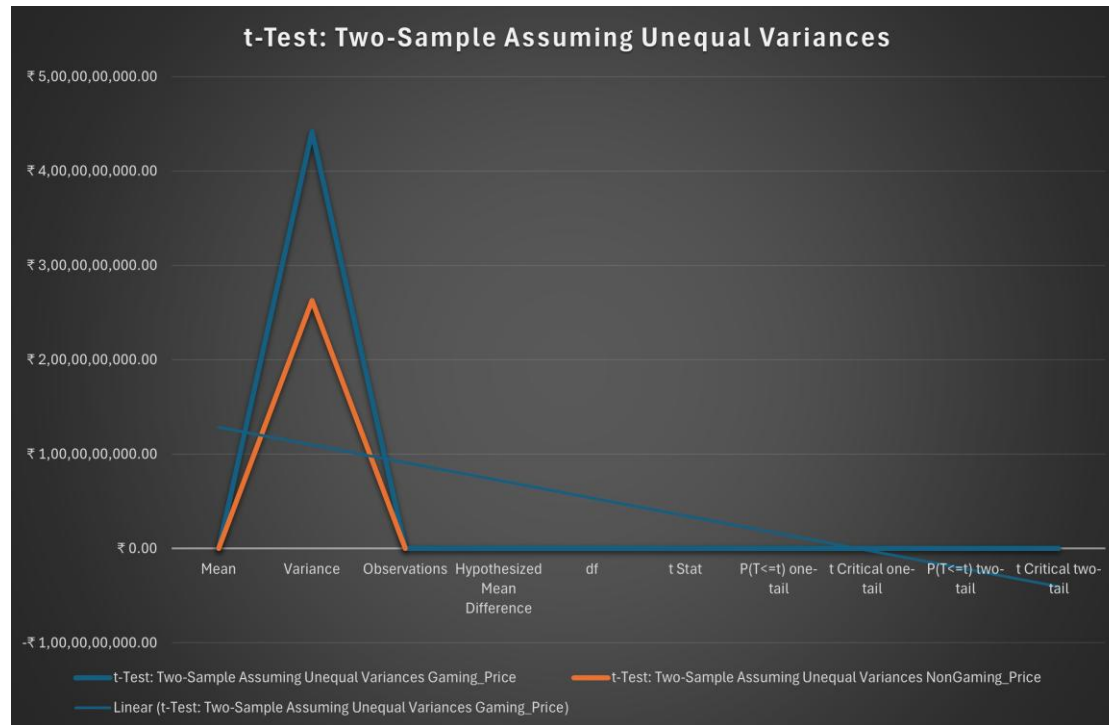
Structured data: Main laptop table (CSV/Excel) with numeric and categorical fields (price, brand, RAM, SSD, rating, etc.).

Semi-structured data: Textual description fields in the original uncleaned dataset (product names and detailed specs strings).

How data is accessed:

Downloaded as CSV from Kaggle → opened and cleaned in Excel and Python → imported into SQL tables → reused in Excel dashboards and Python models.

Basic Statistical Analysis – Price & Gaming vs Non-Gaming



Calculated minimum, maximum, mean, median, mode and standard deviation for laptop **Price** using Excel's Descriptive Statistics tool.

Overall **average price** $\approx$ ₹79,901 and **median** $\approx$ ₹64,990, showing a right-skewed price distribution with some very high-end laptops.

Prices range from ₹9,499 to ₹396,990; the large **standard deviation** ($\sim$ ₹55,880) and variance indicate that prices are highly dispersed.

Performed a **two-sample t-test (unequal variances)** comparing **Gaming_Price** and **NonGaming_Price**.

Mean gaming laptop price $\approx$ ₹108,954 vs non-gaming $\approx$ ₹73,690.

The t-test shows a **very small p-value** ($\approx$ 0), so we **reject the null hypothesis** that the two means are equal.

Conclusion: Gaming laptops are statistically significantly more expensive than non-gaming laptops in this dataset.

Descriptive Analysis (Excel)

Built multiple Pivot Tables on the cleaned laptop dataset (Brand, Price, RAM_GB, SSD_GB, spec_score, Gaming_Flag, Release_Year).

Analyzed key metrics such as average price, average spec_score and count of laptops by brand, RAM size, SSD capacity and release year.

Created PivotCharts (column/bar charts) to visualize:

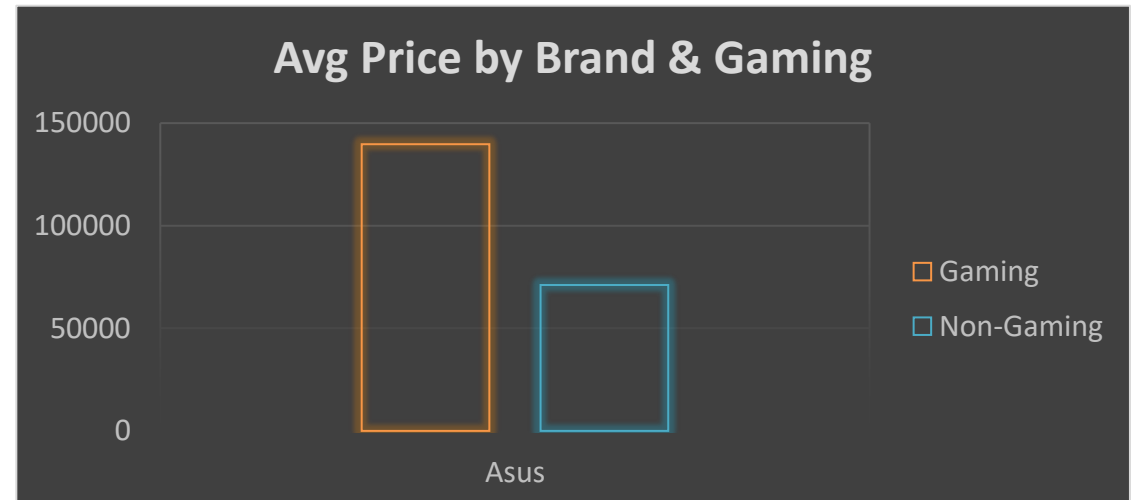
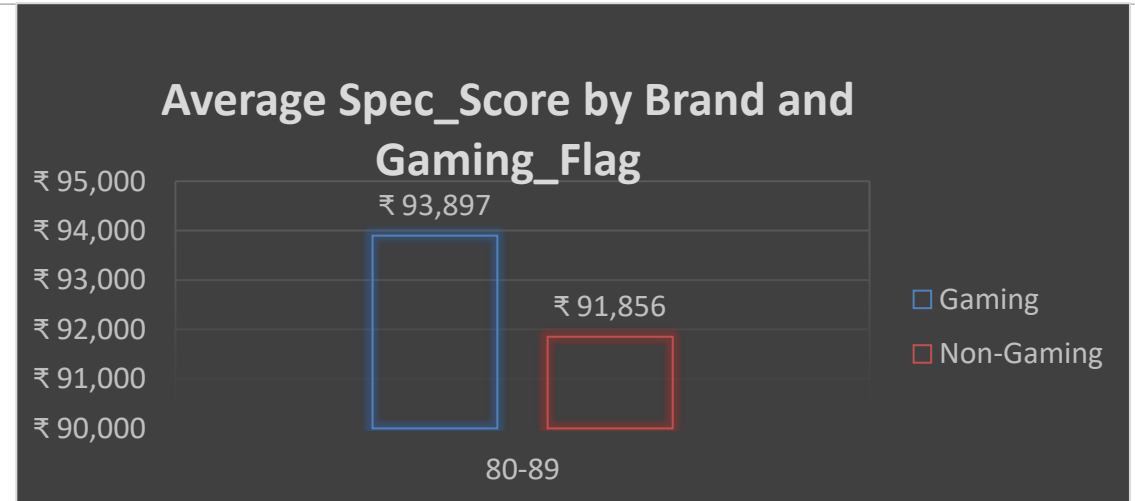
- Average price by brand.

- Number of laptops by release year.

- Average spec_score for gaming vs non-gaming laptops.

Designed a dynamic dashboard sheet in Excel where all charts are controlled by slicers.

Added slicers for Brand, RAM_GB, Gaming_Flag and Release_Year to interactively filter PivotTables and charts for quick descriptive insights.



Database Creation (SQL)

Created a new MySQL database called laptop_dss for the laptop decision-support system.

Defined a single analytical table Laptops with key fields:

laptop_id (PK), name, brand, price, spec_score, ram_gb, release_year, screen_inch, gaming_flag.

Inserted multiple laptops from different brands (HP, Lenovo, Asus, Dell), covering both **gaming** and **non-gaming** segments and various RAM and screen sizes.

Wrote SQL queries to:

Calculate **average price by brand**.

Compare **average spec_score for gaming vs non-gaming** laptops.

Analyze **average price and spec_score by RAM_GB**.

Count laptops by **release_year** and by **brand × gaming_flag**.

List **top laptops by spec_score** and compute a **value score** (spec_score per 1,000 currency units).

Exported key query results to Excel and converted them into graphs, which are used in the static SQL dashboard in the presentation.

```
SELECT release_year, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY release_year
ORDER BY release_year;
```

release_year	laptop_count
2021	1
2022	1
2023	1
2024	8

```
INSERT INTO Laptops
(laptop_id, name, brand, price, spec_score, ram_gb, release_year,
screen_inch, gaming_flag)
VALUES
(9, 'HP 15-fd0186TU', 'HP', 52000, 76.0, 8, 2023, 15.6, 'Non-Gaming'),
(10, 'Lenovo IdeaPad 3 14', 'Lenovo', 45000, 77.0, 8, 2022, 14.0, 'Non-Gaming'),
(11, 'Asus Vivobook 15 2021', 'Asus', 43000, 75.0, 8, 2021, 15.6, 'Non-Gaming');
```

```
SELECT release_year, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY release_year
ORDER BY release_year;
```

release_year	laptop_count
2021	1
2022	1
2023	2
2024	8

SQL-Queries

Query No.	Query Title	SQL Query	Objective	Explanation
1	Full Laptop Dataset	SELECT * FROM Laptops;	Provide a complete view of all laptops available in the dataset.	Lists every record from the Laptops table, including name, brand, price, spec score, RAM, release year, screen size, and gaming flag. Serves as the base dataset for all further analysis.
2	Average Price by Brand	SELECT brand, AVG(price) AS avg_price FROM Laptops GROUP BY brand;	Understand how different brands are positioned in terms of average laptop price.	Groups data by brand and calculates average price, highlighting premium versus budget brands.
3	Average Spec Score by Brand	SELECT brand, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY brand;	Compare technical performance across brands.	Computes average spec score per brand to identify performance-oriented manufacturers.
4	Average Spec Score by Gaming Flag	SELECT gaming_flag, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY gaming_flag;	Test whether gaming laptops have better specifications.	Compares average spec scores between Gaming and Non-Gaming laptops to confirm performance differences.
5	Average Price and Spec Score by RAM Size	SELECT ram_gb, AVG(price) AS avg_price, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY ram_gb ORDER BY ram_gb;	Analyse the impact of RAM size on price and performance.	Shows how increasing RAM correlates with higher prices and better specifications.
6	Laptop Count by Release Year	SELECT release_year, COUNT(*) AS laptop_count FROM Laptops GROUP BY release_year ORDER BY release_year;	Assess how recent the laptop portfolio is.	Counts laptops by release year to evaluate dataset freshness and market relevance.
7	Brand by Gaming Flag Matrix	SELECT brand, gaming_flag, COUNT(*) AS laptop_count FROM Laptops GROUP BY brand, gaming_flag ORDER BY brand, gaming_flag;	Evaluate brand focus on gaming vs non-gaming segments.	Displays how each brand distributes its laptops between gaming and mainstream categories.
8	Cheapest Gaming Laptops	SELECT * FROM Laptops WHERE gaming_flag = 'Gaming' ORDER BY price LIMIT 5;	Identify budget-friendly gaming laptops.	Filters gaming laptops and lists the cheapest options, useful for cost-sensitive buyers.
9	Value-for-Money Ranking	SELECT name, brand, price, spec_score, ROUND(spec_score / (price / 1000), 2) AS score_per_1000 FROM Laptops ORDER BY score_per_1000 DESC;	Rank laptops by performance relative to price.	Uses a derived metric to highlight laptops offering the best performance per cost unit.
10	High-Spec Brands	SELECT brand, AVG(spec_score) AS avg_spec_score FROM Laptops GROUP BY brand HAVING AVG(spec_score) > 80;	Identify consistently high-performance brands.	Filters brands whose average spec score exceeds a defined threshold, identifying premium brands.

Database Creation (SQL) _EER Diagram

Use this as the single slide content next to your EER image:

Title:

MySQL EER Diagram – Laptops Schema

EER diagram shows the **laptops** table that stores one row per laptop model in my dataset.

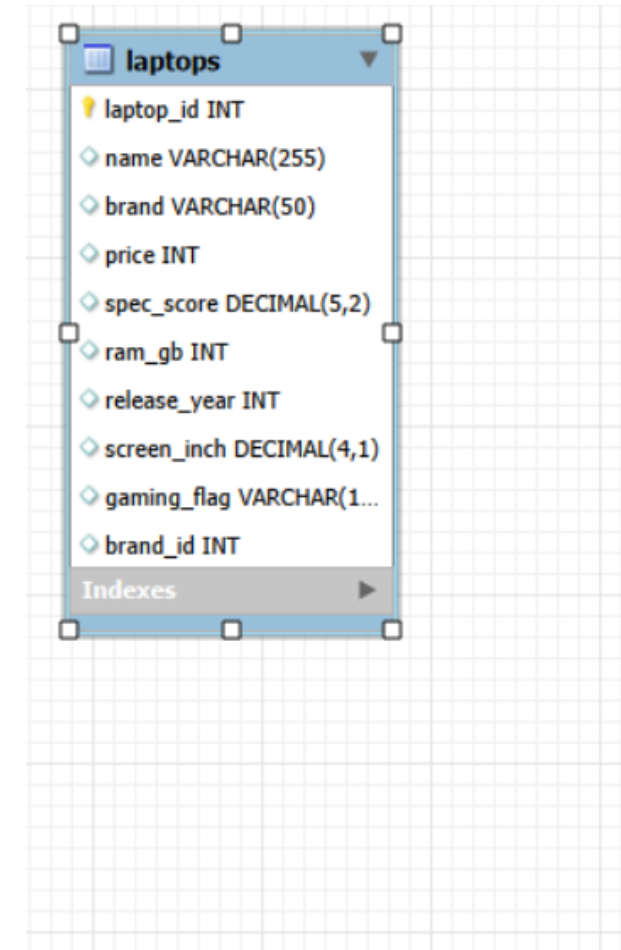
laptop_id is the **primary key** and uniquely identifies each laptop.

Main attributes captured:

Descriptive: name, brand, gaming_flag

Technical/numeric: price, spec_score, ram_gb, release_year, screen_inch

This structure organizes all important laptop information in one place and makes it easy to run SQL analysis, such as comparing brands and calculating average price or spec score.



Normalization – From Raw Table to 3NF

Started with one large unnormalized laptop table where brand, OS and processor details were repeated in every row.

Applied 1NF: made all attributes atomic (no comma-separated features), defined a primary key (LaptopID) and organized the data into a clean table.

Applied 2NF: split the data into separate tables such as BRAND, OPERATING_SYSTEM and LAPTOP, removing partial dependencies and reducing redundancy.

Applied 3NF: created a separate PROCESSOR table and linked it to the LAPTOP table using foreign keys, removing transitive dependencies.

Final structure:

BRAND (BrandID, BrandName, BrandCountry)

OPERATING_SYSTEM (OSID, OSName, Developer)

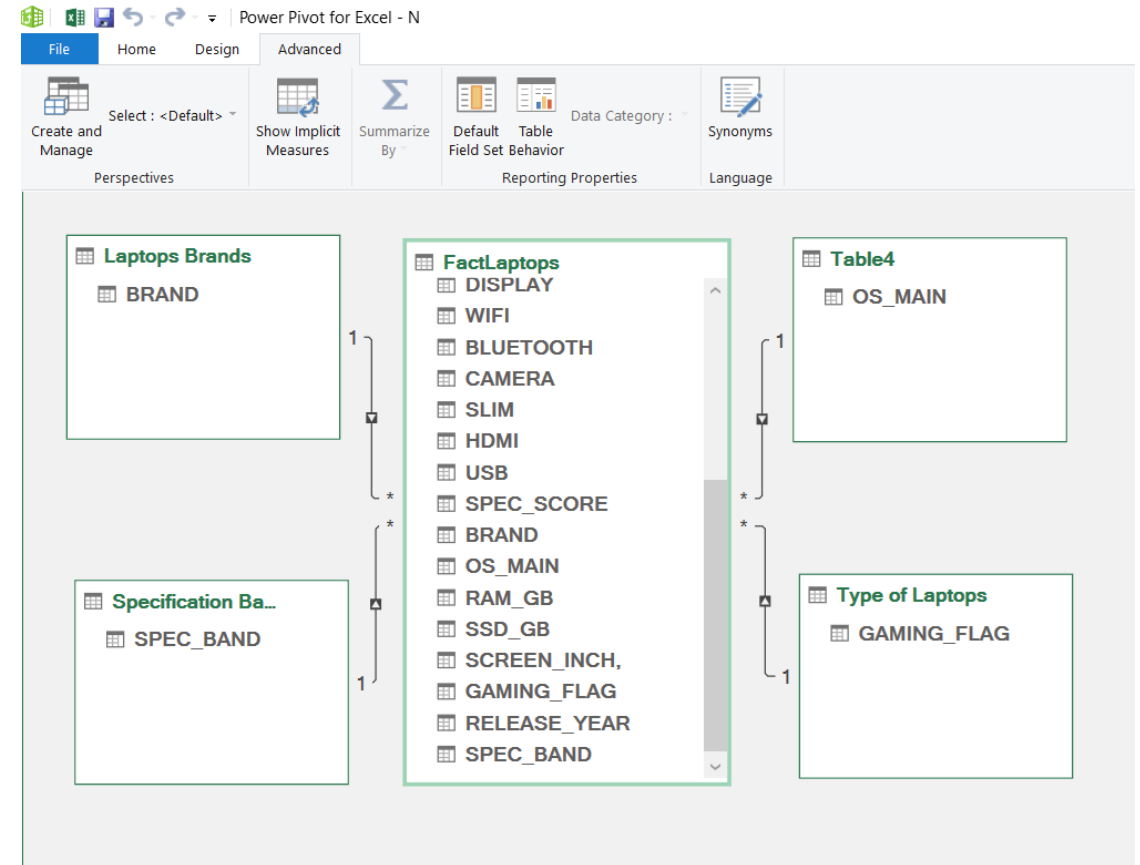
PROCESSOR (ProcessorID, ProcessorName, Type, Cores)

LAPTOP (LaptopID, LaptopName, Price, BrandID, OSID, ProcessorID, RAMGB, SSDGB, Screen, Rating)

Benefits: minimal redundancy, easier updates, consistent master data for brands/OS/processors and a solid foundation for SQL queries and analytics.

Normalization – Laptop Data Model in Power Pivot

- Built a star-schema model in Excel Power Pivot to analyze laptops.
- Central fact table FactLaptops holds all numeric and detailed fields (price, ratings, RAM, SSD, screen size, spec score, release year, gaming flag, etc.)
- Four dimension tables surround it: Laptops Brands, OS_MAIN (Table4), Specification Band, and Type of Laptops
- Each dimension has a one-to-many relationship to FactLaptops (one brand/OS/spec band/type → many laptops)
- This design makes PivotTables easier to use, faster, and more flexible for slicing results by brand, OS, performance band, and gaming vs non-gaming



Predictive Analytics (Python)

Built a Random Forest regression model in Python to predict laptop **Price** using features such as Brand_ID, OS_ID, spec_score, Gaming_Flag, Release_Year, RAM_GB, SSD_GB and Screen_Inch.

Trained the model with 200 trees and evaluated performance on a separate test set.

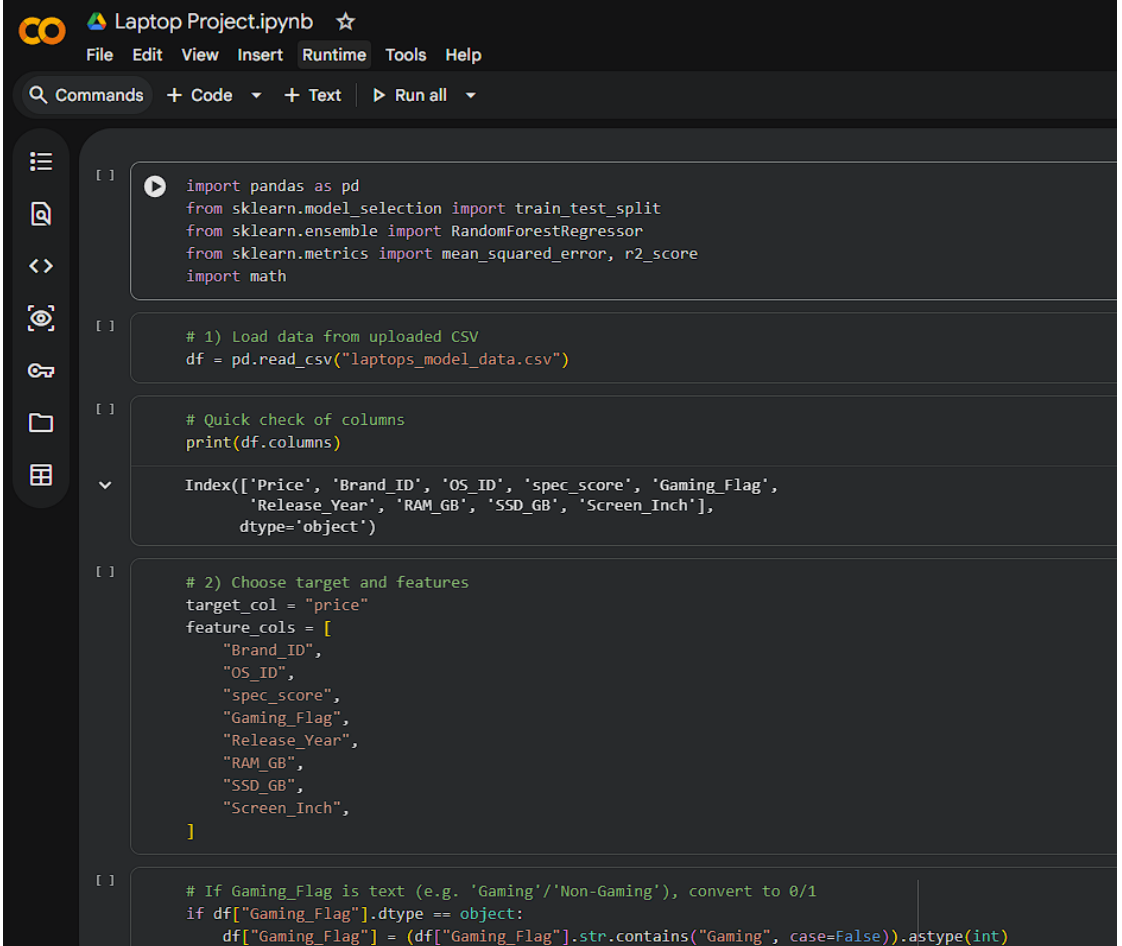
Test RMSE: 28,006 – typical prediction error in currency units for a laptop.

Test R²: 0.67 – the model explains about 67% of the variation in laptop prices.

Average absolute error: 8,067 and **median absolute error:** 4,945 – most predicted prices are within a few thousand units of the true price.

Average percentage error: around 11%
, meaning predicted prices are typically within $\pm 11\%$ of actual prices.

Used the actual vs predicted price table and error metrics to create graphs (e.g., Actual vs Predicted scatter, error distribution) for the Python static dashboard.



```
[ ] ▶ import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import math

[ ] # 1) Load data from uploaded CSV
df = pd.read_csv("laptops_model_data.csv")

[ ] # Quick check of columns
print(df.columns)

Index(['Price', 'Brand_ID', 'OS_ID', 'spec_score', 'Gaming_Flag',
       'Release_Year', 'RAM_GB', 'SSD_GB', 'Screen_Inch'],
      dtype='object')

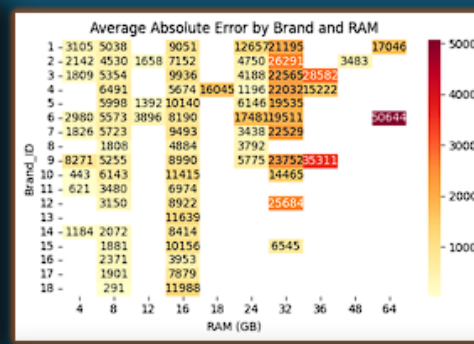
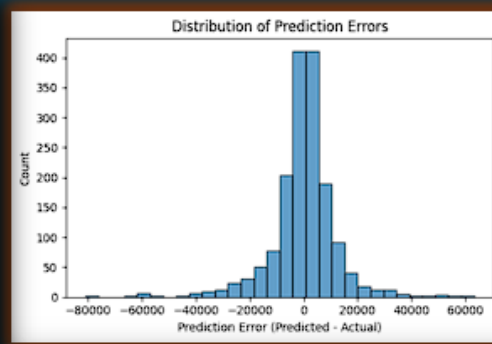
[ ] # 2) Choose target and features
target_col = "price"
feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]

[ ] # If Gaming_Flag is text (e.g. 'Gaming'/'Non-Gaming'), convert to 0/1
if df["Gaming_Flag"].dtype == object:
    df["Gaming_Flag"] = (df["Gaming_Flag"].str.contains("Gaming", case=False)).astype(int)
```

Predictive Analytics (Python) Static Dashboard

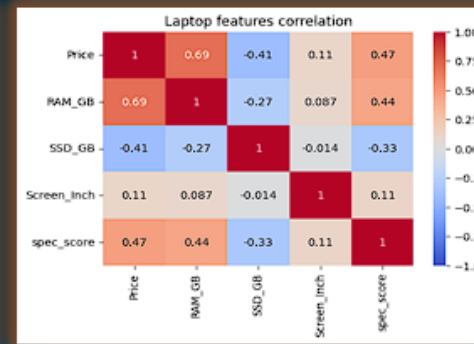
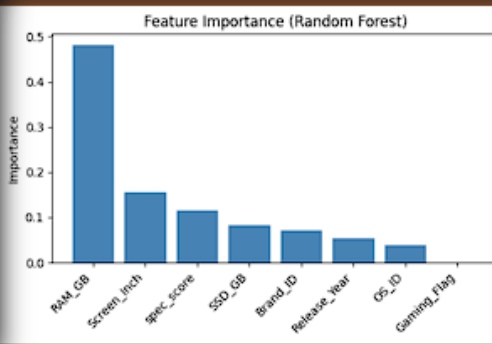
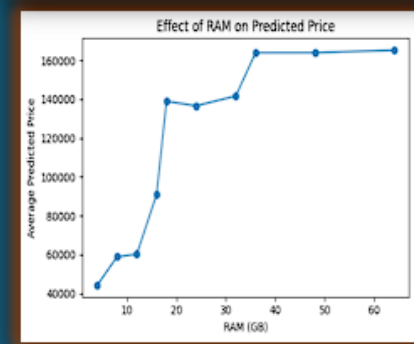


Static Laptop Market Overview Dashboard



Gaming Laptops-288

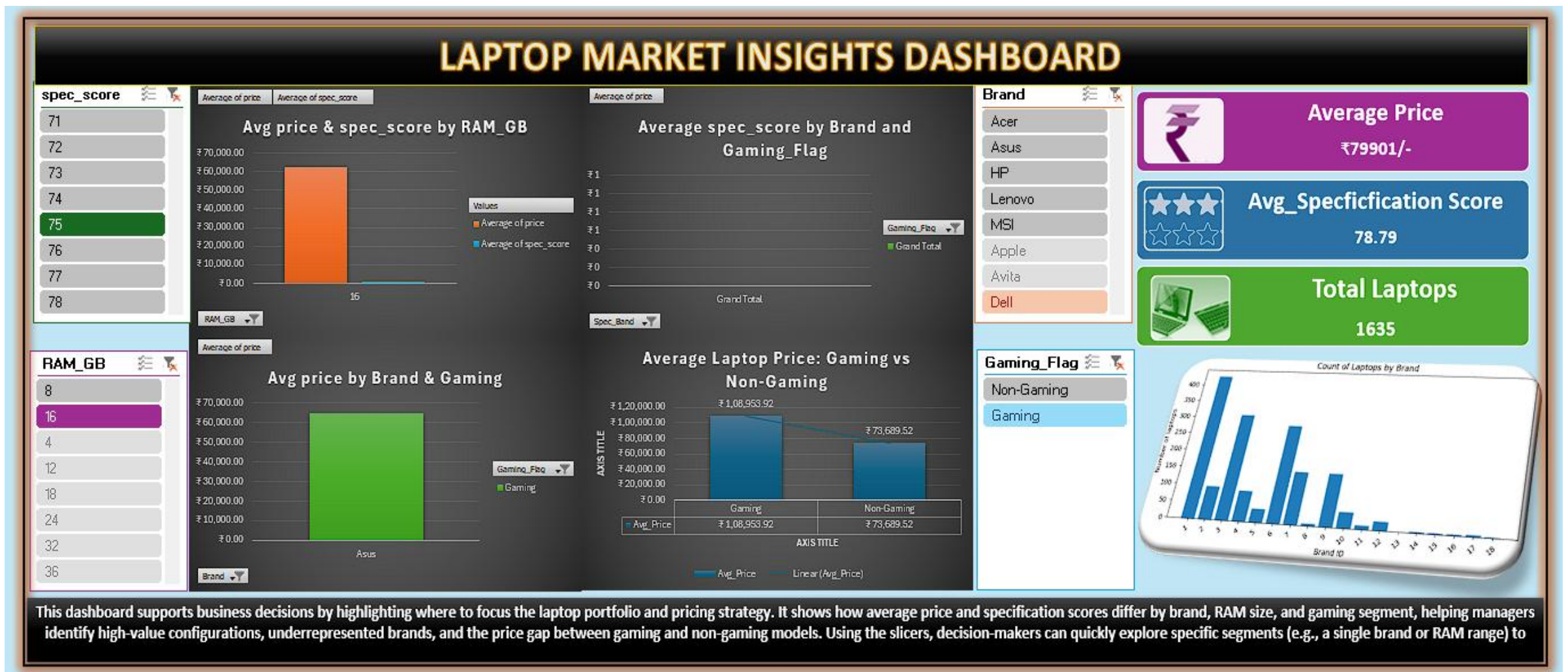
Non Gaming-1347



Costliest Laptop-Gaming

Price-₹3,96,990/-

Dynamic Dashboard



Conclusion & Future AI Integration

Analyzed a large laptop dataset using Excel, SQL and Python to understand how price is influenced by brand, specifications, gaming vs non-gaming and release year.

Descriptive statistics showed highly dispersed and right-skewed prices, with gaming laptops significantly more expensive than non-gaming models.

Excel dashboards, SQL queries and normalized database design provided clear insights into price patterns, brand positioning and product mix.

The Random Forest model achieved good predictive performance, showing that price can be reasonably estimated from a small set of key features.

Future AI steps for the laptop business:

- Deploy the price prediction model as an automated **pricing assistant** to suggest optimal price ranges for new or existing laptops.

- Build a **recommender system** that suggests laptops to customers based on budget, usage (gaming, study, work) and preferred specs.

- Use **customer segmentation (clustering)** on purchase and browsing data to design targeted offers for students, gamers and professionals.

- Integrate **intelligent dashboards** that continuously update KPIs (sales, price competitiveness, spec_score vs price) and highlight anomalies such as underpriced or over-priced models.

- In the long term, combine these AI components into an **AI-driven decision support system** that helps management decide which models to stock, how to price them, and which segments to focus on.

Future AI Agents for the Laptop Business

Agent	Performance Measure	Environment	Actuators	Sensors
Laptop Recommendation Agent	Accuracy of recommendations, customer satisfaction, match with budget & usage, response time	Cleaned laptop dataset (Excel), customer preferences, recommendation interface	Display recommended laptops, rank options by value-for-money, filter by budget/specs	User inputs (budget, usage: student/gamer/professional), laptop specs (RAM_GB, SSD_GB, spec_score), price
Laptop Price Analysis Agent	Correct price classification, statistical accuracy, processing speed	Laptop price & specification dataset (Excel / SQL)	Generate price insights, classify laptops (budget / mid / premium), show trends	Laptop prices, brand, spec_score, gaming_flag, release_year
Laptop Specification Comparison Agent	Comparison accuracy, clarity, completeness of specs	Laptop dataset, selected laptops for comparison	Show comparison tables, highlight best specs, rank laptops	Laptop attributes (RAM_GB, SSD_GB, Screen_Inch, OS_Type, processor family)
Pricing Prediction Agent	Prediction accuracy (RMSE, R^2), error rate	Python Random Forest model, historical laptop data	Predict laptop prices, suggest optimal price range	Brand_ID, OS_ID, spec_score, RAM_GB, SSD_GB, Screen_Inch, gaming_flag
Gaming vs Non-Gaming Analysis Agent	Statistical validity, insight quality	Excel & SQL analytical environment	Generate t-test results, charts, insights	Gaming_Flag, laptop prices, spec_score
Dashboard Monitoring Agent	Timeliness of alerts, KPI accuracy	Excel Dynamic Dashboard	Refresh dashboards, highlight anomalies	PivotTables, slicer selections, KPI thresholds
Manager Insights Agent	Relevance of insights, response clarity	SQL database + Excel summaries	Answer business questions, generate reports	SQL query results, aggregated metrics
Customer Segmentation Agent (Future)	Clustering quality, segment interpretability	Python analytics environment	Assign customer segments, recommend offers	Purchase patterns, preferred specs, budget range

Thank you for your attention

macromedia-fachhochschule.de

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import math
```

```
# 1) Load data from uploaded CSV
df = pd.read_csv("laptops_model_data.csv")
```

```
# Quick check of columns
print(df.columns)
```

```
Index(['Price', 'Brand_ID', 'OS_ID', 'spec_score', 'Gaming_Flag',
       'Release_Year', 'RAM_GB', 'SSD_GB', 'Screen_Inch'],
      dtype='object')
```

```
# 2) Choose target and features
target_col = "price"
feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]
```

```
# If Gaming_Flag is text (e.g. 'Gaming'/'Non-Gaming'), convert to 0/1
if df["Gaming_Flag"].dtype == object:
    df["Gaming_Flag"] = (df["Gaming_Flag"].str.contains("Gaming", case=False)).astype(int)

X = df[feature_cols]
y = df[target_col]

# 3) Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'price'
```

The above exception was the direct cause of the following exception:

```
-----
KeyError                                Traceback (most recent call last)
-----
      2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
    3813     except TypeError:
    3814         # If we have a listlike key, _check_indexing_error will raise

KeyError: 'price'
```

```
df = pd.read_csv("laptops_model_data.csv")
print(df.columns)
df.head()
```

```
Index(['Price', 'Brand_ID', 'OS_ID', 'spec_score', 'Gaming_Flag',
      'Release_Year', 'RAM_GB', 'SSD_GB', 'Screen_Inch'],
      dtype='object')
```

	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	SSD_GB	Screen_Inch
0	31990	1	1	74.0	Non-Gaming	2025	8	512	14.0
1	17990	2	1	NaN	Non-Gaming	2025	8	256	11.6
2	41500	1	1	74.0	Non-Gaming	2025	16	512	15.6
3	63990	2	1	81.0	Non-Gaming	2025	12	512	15.6
4	77990	1	2	81.0	Non-Gaming	2024	16	256	13.6

```
target_col = "price"
```

```
target_col = "Price"
# or
target_col = "price_inr"
```

```
feature_cols = [
    "Brand_ID",      # or "brand_id"
    "OS_ID",         # or "os_id"
    "spec_score",    # check exact spelling
    "Gaming_Flag",   # or "gaming_flag"
    "Release_Year",  # or "release_year"
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]
```

```
import pandas as pd
```

```
df = pd.read_csv("laptops_model_data.csv")
print(df.columns.tolist())
print(df.head(3))
```

```
['Price', 'Brand_ID', 'OS_ID', 'spec_score', 'Gaming_Flag', 'Release_Year', 'RAM_GB', 'SSD_GB', 'Screen_Inch']
```

	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	\
0	31990	1	1	74.0	Non-Gaming	2025	8	
1	17990	2	1	NaN	Non-Gaming	2025	8	
2	41500	1	1	74.0	Non-Gaming	2025	16	

	SSD_GB	Screen_Inch
0	512	14.0
1	256	11.6
2	512	15.6

```
df = pd.read_csv("laptops_model_data.csv")

# Add a serial number column starting at 1
df.insert(0, "S_No", range(1, len(df) + 1))

print(df.head(3))
```

	S_No	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	\
0	1	31990	1	1	74.0	Non-Gaming	2025	8	
1	2	17990	2	1	NaN	Non-Gaming	2025	8	
2	3	41500	1	1	74.0	Non-Gaming	2025	16	

	SSD_GB	Screen_Inch
0	512	14.0
1	256	11.6
2	512	15.6

```
df.to_csv("laptops_model_data_with_sno.csv", index=False)
```

```
target_col = "Price"
feature_cols = ["Brand_ID", "OS_ID", "spec_score", "Gaming_Flag",
               "Release_Year", "RAM_GB", "SSD_GB", "Screen_Inch"]
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import math
```

```
# 1) Load data
df = pd.read_csv("laptops_model_data.csv")
```

```
# Optional: add S_No starting at 1 (only for readability)
df.insert(0, "S_No", range(1, len(df) + 1))
```

```
# 2) Target and features
target_col = "Price"
feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]
```

```
# Convert Gaming_Flag text to 0/1
if df["Gaming_Flag"].dtype == object:
    df["Gaming_Flag"] = (df["Gaming_Flag"].str.contains("Gaming", case=False)).astype(int)
    X = df[feature_cols]
    y = df[target_col]

# 3) Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 4) Train model
model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train, y_train)
```

▼ **RandomForestRegressor** ⓘ ?

RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42)

```
# 5) Evaluate
y_pred = model.predict(X_test)
rmse = math.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Test RMSE:", rmse)
print("Test R^2:", r2)
```

```
Test RMSE: 27661.00821591
Test R^2: 0.6820797089798698
```

```
# 6) One example prediction
sample = X_test.iloc[[0]]
print("Sample features:\n", sample)
print("Actual price:", float(y_test.iloc[0]))
print("Predicted price:", float(model.predict(sample)[0]))
```

```
Sample features:
   Brand_ID  OS_ID  spec_score  Gaming_Flag  Release_Year  RAM_GB  SSD_GB  \
937         4      1         77.0           1           2023      16      512

   Screen_Inch
937         15.6
Actual price: 55990.0
Predicted price: 60718.751249999994
```

```
# Handle missing spec_score: fill NaN with column mean
spec_mean = df["spec_score"].mean()
df["spec_score"] = df["spec_score"].fillna(spec_mean)
```

```
print("Filled spec_score NaN with mean:", spec_mean)
```

```
Filled spec_score NaN with mean: 78.79612468407751
```

```
# Simple CPU strength proxy (0 = low, 1 = mid, 2 = high)
def cpu_strength(row):
    # Very rough rule of thumb
    if row["RAM_GB"] >= 16:
        return 2
    elif row["RAM_GB"] >= 8:
        return 1
    else:
        return 0
df["CPU_strength"] = df.apply(cpu_strength, axis=1)
```

```
feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
    "CPU_strength",
]
```

```
print("Test RMSE:", rmse)
print("Test R^2:", r2)
```

```
Test RMSE: 27661.00821591
Test R^2: 0.6820797089798698
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import math

# 1) Load data
df = pd.read_csv("laptops_model_data.csv")

# Optional: add S_No starting at 1 (only for readability)
df.insert(0, "S_No", range(1, len(df) + 1))

# 2) Handle missing spec_score
spec_mean = df["spec_score"].mean()
df["spec_score"] = df["spec_score"].fillna(spec_mean)
print("Filled spec_score NaN with mean:", spec_mean)

# 3) Convert Gaming_Flag text to 0/1
if df["Gaming_Flag"].dtype == object:
    df["Gaming_Flag"] = (df["Gaming_Flag"].str.contains("Gaming", case=False)).astype(int)

# 4) Add CPU_strength feature
def cpu_strength(row):
    if row["RAM_GB"] >= 16:
        return 2
    elif row["RAM_GB"] >= 8:
        return 1
    else:
        return 0

df["CPU_strength"] = df.apply(cpu_strength, axis=1)

# 5) Target and features
target_col = "Price"
feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
    "CPU_strength",
```



```

]

X = df[feature_cols]
y = df[target_col]

# 6) Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Filled spec_score NaN with mean: 78.79612468407751

```

# 7) Train model
model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train, y_train)

# 8) Evaluate
y_pred = model.predict(X_test)
rmse = math.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Test RMSE:", rmse)
print("Test R^2:", r2)

# 9) Example prediction
sample = X_test.iloc[[0]]
print("Sample features:\n", sample)
print("Actual price:", float(y_test.iloc[0]))
print("Predicted price:", float(model.predict(sample)[0]))

```

Test RMSE: 28006.004000416826
 Test R^2: 0.6740998735153796
 Sample features:

	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	SSD_GB	\
937	4	1	77.0	1	2023	16	512	

Screen_Inch CPU_strength
 937 15.6 2
 Actual price: 55990.0
 Predicted price: 62817.53166666667

```

print("Test RMSE:", rmse)
print("Test R^2:", r2)

```

Test RMSE: 28006.004000416826
 Test R^2: 0.6740998735153796

```

# Retrain on full dataset (all rows)
X_full = df[feature_cols]
y_full = df[target_col]

final_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
final_model.fit(X_full, y_full)

# Predictions for all laptops
df["Predicted_Price"] = final_model.predict(X_full)

```

```

output_cols = [
    "Price",
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
    "Predicted_Price",
]

```

```
df[output_cols].to_csv("laptop_predictions.csv", index=False)
print("Saved laptop_predictions.csv")
```

Saved laptop_predictions.csv

```
# 1) Retrain on full dataset
X_full = df[feature_cols]
y_full = df[target_col]

from sklearn.ensemble import RandomForestRegressor

final_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
final_model.fit(X_full, y_full)

# 2) Add Predicted_Price column
df["Predicted_Price"] = final_model.predict(X_full)

# 3) Save a new CSV
df.to_csv("laptop_predictions.csv", index=False)
print("Saved laptop_predictions.csv")
```

Saved laptop_predictions.csv

```
df.to_csv("laptop_predictions.csv", index=False)
```

```
# 1) Make sure df is loaded and has features + trained model

# If needed, reload:
df = pd.read_csv("laptops_model_data.csv")

# Recreate all preprocessing
df["spec_score"] = df["spec_score"].fillna(df["spec_score"].mean())
if df["Gaming_Flag"].dtype == object:
    df["Gaming_Flag"] = (df["Gaming_Flag"].str.contains("Gaming", case=False)).astype(int)

feature_cols = [
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]

X_full = df[feature_cols]
y_full = df["Price"]

from sklearn.ensemble import RandomForestRegressor

final_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
final_model.fit(X_full, y_full)

# 2) Add predictions and RESAVE
df["Predicted_Price"] = final_model.predict(X_full)

df.to_csv("laptop_predictions.csv", index=False)
print("Saved laptop_predictions.csv with Predicted_Price")
```

Saved laptop_predictions.csv with Predicted_Price

```
df.head()
```

	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	SSD_GB	Screen_Inch	Predicted_Price
0	31990	1	1	74.000000	1	2025	8	512	14.0	33653.010405
1	17990	2	1	78.796125	1	2025	8	256	11.6	22532.895000
2	41500	1	1	74.000000	1	2025	16	512	15.6	48576.935000
3	63990	2	1	81.000000	1	2025	12	512	15.6	62332.307333
4	77990	1	2	81.000000	1	2024	16	256	13.6	83784.700000

```
# Error per row (signed and absolute)
df["Error"] = df["Predicted_Price"] - df["Price"]
df["Abs_Error"] = df["Error"].abs()
df["Pct_Error"] = df["Abs_Error"] / df["Price"] * 100

# Quick check of first rows
df[["Price", "Predicted_Price", "Error", "Abs_Error", "Pct_Error"]].head()
```

	Price	Predicted_Price	Error	Abs_Error	Pct_Error
0	31990	33653.010405	1663.010405	1663.010405	5.198532
1	17990	22532.895000	4542.895000	4542.895000	25.252335
2	41500	48576.935000	7076.935000	7076.935000	17.052855
3	63990	62332.307333	-1657.692667	1657.692667	2.590550
4	77990	83784.700000	5794.700000	5794.700000	7.430055

```
print("Average absolute error:", df["Abs_Error"].mean())
print("Median absolute error:", df["Abs_Error"].median())
print("Max absolute error:", df["Abs_Error"].max())

print("Average % error:", df["Pct_Error"].mean())
print("Median % error:", df["Pct_Error"].median())
print("Max % error:", df["Pct_Error"].max())
```

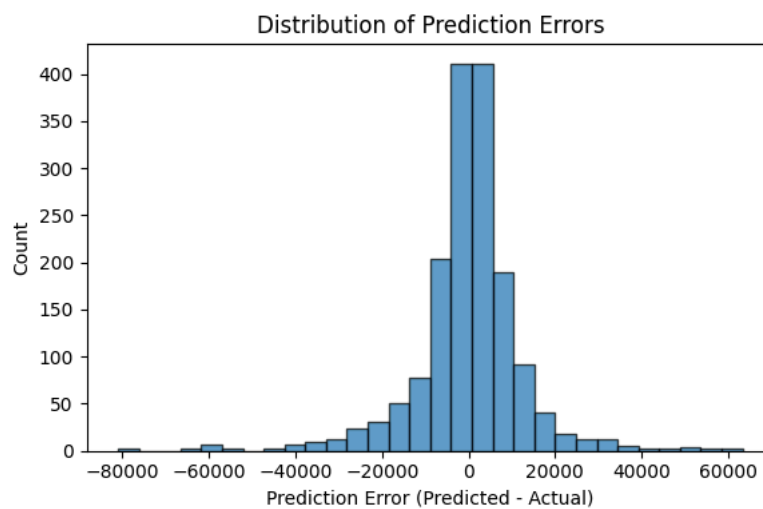
```
Average absolute error: 8066.726779669112
Median absolute error: 4945.082499999997
Max absolute error: 81016.01361471863
Average % error: 11.049338901734282
Median % error: 7.7414489215581135
Max % error: 169.92785676671934
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Laptop Prices")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=300)
plt.show()
```



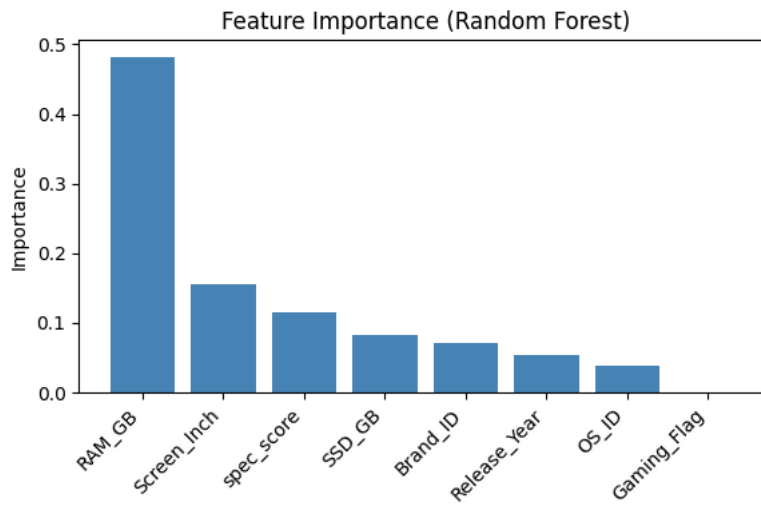
```
plt.figure(figsize=(6,4))
plt.hist(df["Error"], bins=30, edgecolor="black", alpha=0.7)
plt.xlabel("Prediction Error (Predicted - Actual)")
plt.ylabel("Count")
plt.title("Distribution of Prediction Errors")
plt.tight_layout()
plt.savefig("error_distribution.png", dpi=300)
plt.show()
```



```
import numpy as np

importances = final_model.feature_importances_
indices = np.argsort(importances)[::-1]
names = np.array(feature_cols)[indices]

plt.figure(figsize=(6,4))
plt.bar(range(len(feature_cols)), importances[indices], color="steelblue")
plt.xticks(range(len(feature_cols)), names, rotation=45, ha="right")
plt.ylabel("Importance")
plt.title("Feature Importance (Random Forest)")
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=300)
plt.show()
```



```
num_cols = [
    "Price",
    "Predicted_Price",
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]

corr = df[num_cols].corr()
```

```
num_cols = [
    "Price",
    "Predicted_Price",
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
    "CPU_strength",
]

# Keep only the columns that are present in df
num_cols = [c for c in num_cols if c in df.columns]

corr = df[num_cols].corr()
```

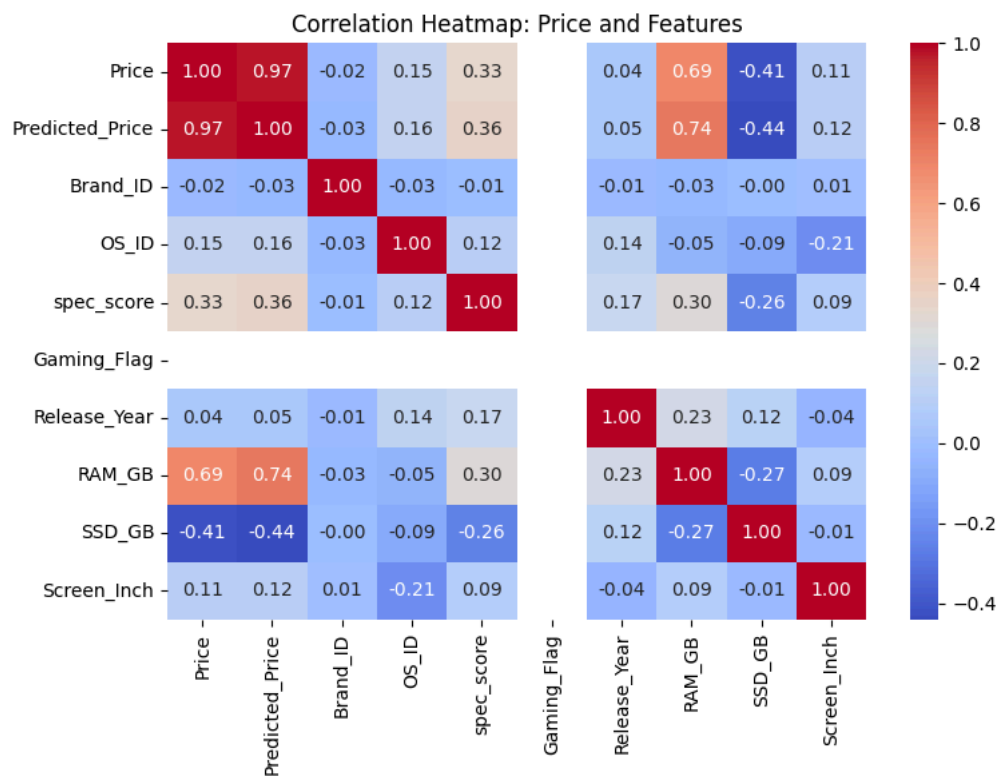
```
import seaborn as sns
import matplotlib.pyplot as plt

# Only columns that exist
num_cols = [
    "Price",
    "Predicted_Price",
    "Brand_ID",
    "OS_ID",
    "spec_score",
    "Gaming_Flag",
    "Release_Year",
    "RAM_GB",
    "SSD_GB",
    "Screen_Inch",
]

corr = df[num_cols].corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap: Price and Features")
```

```
plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=300)
plt.show()
```

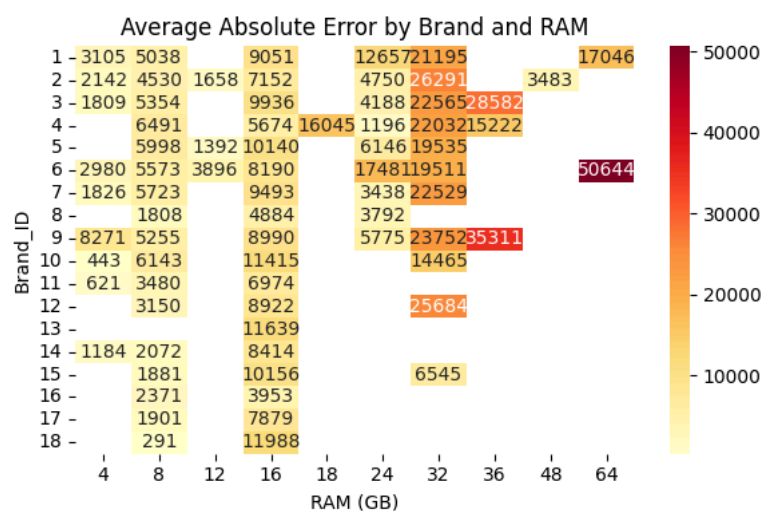


```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Make sure Error exists (from earlier step)
# df["Error"] = df["Predicted_Price"] - df["Price"]

pivot = df.pivot_table(
    values="Abs_Error",          # or "Pct_Error"
    index="Brand_ID",
    columns="RAM_GB",
    aggfunc="mean"
)

plt.figure(figsize=(6,4))
sns.heatmap(pivot, annot=True, fmt=".0f", cmap="YlOrRd")
plt.title("Average Absolute Error by Brand and RAM")
plt.xlabel("RAM (GB)")
plt.ylabel("Brand_ID")
plt.tight_layout()
plt.savefig("error_heatmap_brand_ram.png", dpi=300)
plt.show()
```



```

import numpy as np
import matplotlib.pyplot as plt

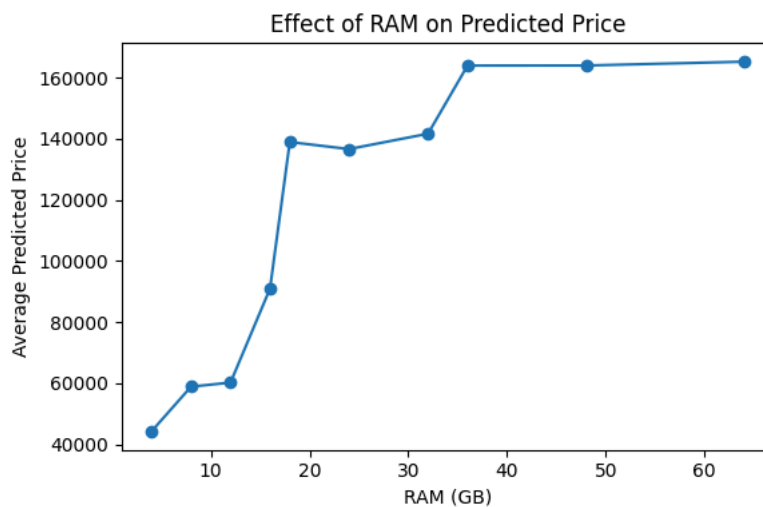
# Choose feature to analyze
feature = "RAM_GB"

# Unique sorted values of RAM
vals = np.sort(df[feature].unique())

avg_preds = []
for v in vals:
    tmp = df.copy()
    tmp[feature] = v
    avg_preds.append(final_model.predict(tmp[feature_cols]).mean())

plt.figure(figsize=(6,4))
plt.plot(vals, avg_preds, marker="o")
plt.xlabel("RAM (GB)")
plt.ylabel("Average Predicted Price")
plt.title("Effect of RAM on Predicted Price")
plt.tight_layout()
plt.savefig("pdp_ram.png", dpi=300)
plt.show()

```



```

summary = f"""
Random forest regression model for laptop prices:
- Test RMSE: {rmse:,.0f}
- Test R²: {r2:.2f}
- Average absolute error: {df['Abs_Error'].mean():,.0f}
- Median absolute error: {df['Abs_Error'].median():,.0f}
- Average % error: {df['Pct_Error'].mean():.1f}%
"""

print(summary)

```

```

Random forest regression model for laptop prices:
- Test RMSE: 28,006
- Test R²: 0.67
- Average absolute error: 8,067
- Median absolute error: 4,945
- Average % error: 11.0%

```

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load your original dataset
df = pd.read_csv("laptops_model_data.csv")

# 2. Keep only numeric columns you care about
num_cols = ["price", "RAMGB", "SSDGB", "ScreenInch", "rating100", "specscore"]
data_num = df[num_cols]

# 3. Correlation matrix (dataset only, no predicted values)
corr = data_num.corr()

```

```
# 4. Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation heatmap of original dataset (before prediction)")
plt.tight_layout()
plt.show()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipython-input-2712248615.py in <cell line: 0>()
      4
      5 # 1. Load your original dataset
----> 6 df = pd.read_csv("laptops_model_data.csv")
      7
      8 # 2. Keep only numeric columns you care about

----- 4 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression,
memory_map, is_text, errors, storage_options)
      871     if ioargs.encoding and "b" not in ioargs.mode:
      872         # Encoding
--> 873         handle = open(
      874             handle,
      875             ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'laptops_model_data.csv'
```

```
import os
print(os.getcwd())
```

```
/content
```

```
df = pd.read_csv(r"/laptops_model_data.csv")
```

```
import os
print(os.listdir())
```

```
['.config', 'sample_data']
```

Double-click (or enter) to edit

```
from google.colab import files
uploaded = files.upload() # choose laptops_model_data.csv from your computer
print(uploaded.keys())   # should show 'laptops_model_data.csv'
import pandas as pd
df = pd.read_csv("laptops_model_data.csv")
df.head()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving laptops_model_data.csv to laptops_model_data.csv
dict_keys(['laptops_model_data.csv'])

	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	SSD_GB	Screen_Inch
0	31990	1	1	74.0	Non-Gaming	2025	8	512	14.0
1	17990	2	1	NaN	Non-Gaming	2025	8	256	11.6
2	41500	1	1	74.0	Non-Gaming	2025	16	512	15.6
3	63990	2	1	81.0	Non-Gaming	2025	12	512	15.6

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

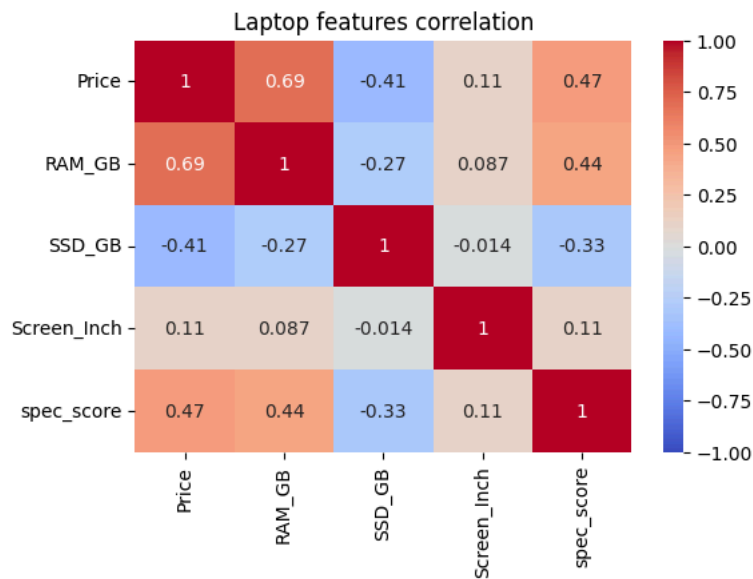
```
# 1) Select numeric columns you care about
num_cols = ["Price", "RAM_GB", "SSD_GB", "Screen_Inch", "spec_score"] # adapt to your exact column names
```

```
# 2) Compute correlation matrix
corr = df[num_cols].corr()
```

```
# 3) Plot heatmap
```



```
plt.figure(figsize=(6,4))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Laptop features correlation")
plt.show()
```



```
from google.colab import files
uploaded = files.upload() # choose laptops_model_data.csv from your computer
print(uploaded.keys()) # should show 'laptops_model_data.csv'
import pandas as pd
df = pd.read_csv("laptops_model_data.csv")
df.head()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving laptops_model_data.csv to laptops_model_data.csv

dict_keys(['laptops_model_data.csv'])

	Price	Brand_ID	OS_ID	spec_score	Gaming_Flag	Release_Year	RAM_GB	SSD_GB	Screen_Inch
0	31990	1	1	74.0	Non-Gaming	2025	8	512	14.0
1	17990	2	1	NaN	Non-Gaming	2025	8	256	11.6
2	41500	1	1	74.0	Non-Gaming	2025	16	512	15.6
3	63990	2	1	81.0	Non-Gaming	2025	12	512	15.6

```
import pandas as pd

df = pd.read_csv("laptops_model_data.csv") # or read_excel("Laptops_Project.xlsx")
```

```
print(df.columns.tolist())
```

```
['Price', 'Brand_ID', 'OS_ID', 'spec_score', 'Gaming_Flag', 'Release_Year', 'RAM_GB', 'SSD_GB', 'Screen_Inch']
```

```
total_laptops = len(df) # KPI 1
avg_price = df["Price"].mean() # KPI 2
avg_spec = df["spec_score"].mean() # KPI 3
```

```
print(total_laptops, avg_price, avg_spec)
```

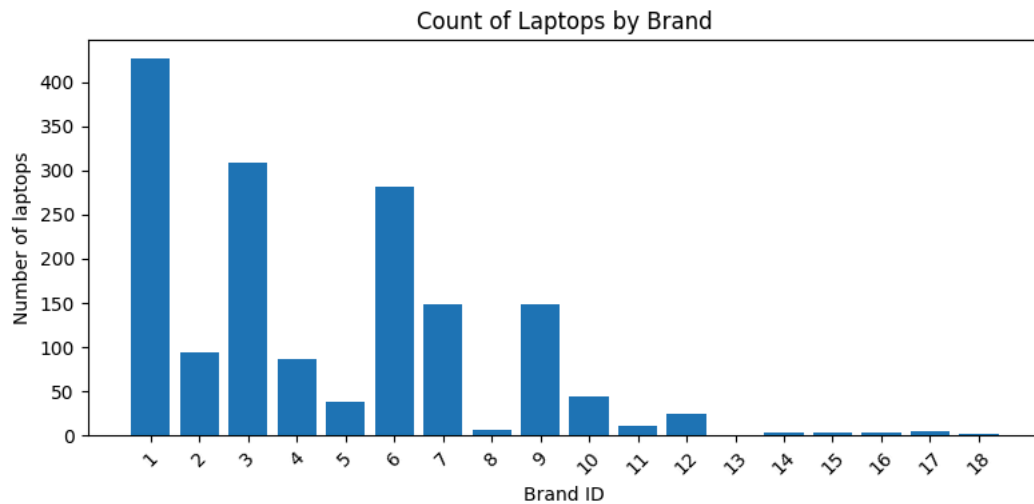
```
1635 79901.22691131498 78.79612468407751
```

```
import matplotlib.pyplot as plt

counts = df["Brand_ID"].value_counts().sort_index()

plt.figure(figsize=(8,4))
plt.bar(counts.index.astype(str), counts.values)
```

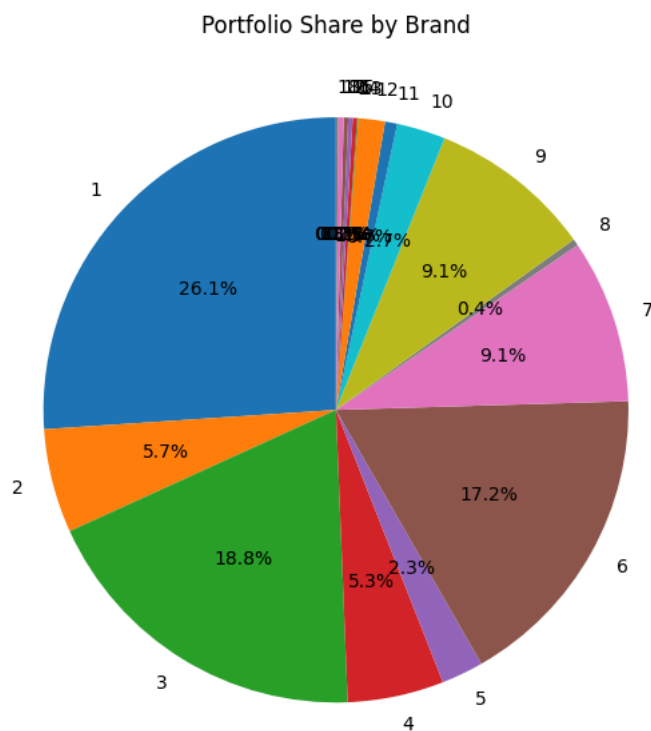
```
plt.xlabel("Brand ID")
plt.ylabel("Number of laptops")
plt.title("Count of Laptops by Brand")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt

brand_counts = df["Brand_ID"].value_counts().sort_index()

plt.figure(figsize=(6,6))
plt.pie(
    brand_counts.values,
    labels=brand_counts.index.astype(str), # later replace with real names
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Portfolio Share by Brand")
plt.tight_layout()
plt.show()
```



```
len(df) # total laptops
df["Gaming_Flag"].value_counts()
df.describe() # numeric summary
```

```
df["Release_Year"].value_counts().sort_index()
```

count	
Release_Year	
2018	3
2019	12
2020	18
2021	33
2022	409
2023	598
2024	409
2025	153

dtype: int64

```
n_total = len(df)
gaming_counts = df["Gaming_Flag"].value_counts()
gaming_pct = (gaming_counts.get("Gaming", 0) / n_total) * 100
nongaming_pct = (gaming_counts.get("Non-Gaming", 0) / n_total) * 100
gaming_counts, gaming_pct, nongaming_pct
```

(Gaming_Flag
Non-Gaming 1347
Gaming 288
Name: count, dtype: int64,
np.float64(17.61467889908257),
np.float64(82.38532110091742))

```
df["Price"].describe()
df[["RAM_GB", "SSD_GB"]].describe()
df.groupby("Gaming_Flag")["spec_score"].mean()
df["Screen_Inch"].value_counts().head()
```

count	
Screen_Inch	
15.6	822
14.0	420
16.0	145
17.3	33
13.6	31

dtype: int64

```
df["Price"].min(), df["Price"].max()
df["RAM_GB"].max(), df["SSD_GB"].max()
df["spec_score"].max()
df["Screen_Inch"].max()
```

18.0

SQL SCRIPT

```
DROP TABLE IF EXISTS Laptops;
DROP TABLE IF EXISTS Brand;
DESCRIBE Laptops;
DROP DATABASE IF EXISTS laptop_dss;
CREATE DATABASE laptop_dss;
USE laptop_dss;
CREATE TABLE Laptops (
    laptop_id INT PRIMARY KEY,
    name VARCHAR(255),
    brand VARCHAR(50),
    price INT,
    spec_score DECIMAL(5,2),
    ram_gb INT,
    release_year INT,
    screen_inch DECIMAL(4,1),
    gaming_flag VARCHAR(10)
);
SHOW TABLES;
INSERT INTO Laptops
(laptop_id, name, brand, price, spec_score, ram_gb, release_year, screen_inch, gaming_flag)
VALUES
(1, 'HP Victus 15-fa1227TX', 'HP', 85000, 84.0, 16, 2024, 15.6, 'Gaming'),
(2, 'Lenovo IdeaPad Slim 3', 'Lenovo', 55000, 77.0, 8, 2024, 15.6, 'Non-Gaming'),
(3, 'Asus Vivobook Go 15', 'Asus', 40000, 78.0, 8, 2024, 15.6, 'Non-Gaming'),
(4, 'Dell G15 5530', 'Dell', 110000, 86.0, 16, 2024, 15.6, 'Gaming');
SELECT * FROM Laptops;
SELECT
    brand,
    AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
SELECT
    gaming_flag,
    AVG(spec_score) AS avg_spec_score
FROM Laptops
GROUP BY gaming_flag;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
SELECT gaming_flag, AVG(spec_score) AS avg_spec_score
FROM Laptops
GROUP BY gaming_flag;
SELECT ram_gb,
    AVG(price) AS avg_price,
    AVG(spec_score) AS avg_spec_score
FROM Laptops
```

```

GROUP BY ram_gb
ORDER BY ram_gb;
SELECT release_year, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY release_year
ORDER BY release_year;
SELECT brand, gaming_flag, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY brand, gaming_flag;
SELECT *
FROM Laptops
WHERE gaming_flag = 'Gaming'
ORDER BY price
LIMIT 5;
SELECT *
FROM Laptops
ORDER BY spec_score DESC
LIMIT 5;
SELECT release_year, AVG(price) AS avg_price
FROM Laptops
WHERE release_year >= 2023
GROUP BY release_year
ORDER BY release_year;
SELECT name,
       brand,
       price,
       spec_score,
       ROUND(spec_score / (price / 1000), 2) AS score_per_1000
FROM Laptops
ORDER BY score_per_1000 DESC;
SELECT brand,
       AVG(spec_score) AS avg_spec_score
FROM Laptops
GROUP BY brand
HAVING AVG(spec_score) > 80;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
INSERT INTO Laptops
(laptop_id, name, brand, price, spec_score, ram_gb, release_year, screen_inch, gaming_flag)
VALUES
(5, 'HP Pavilion 16', 'HP', 78000, 82.0, 16, 2024, 16.0, 'Non-Gaming'),
(6, 'Lenovo LOQ 15', 'Lenovo', 92000, 85.0, 16, 2024, 15.6, 'Gaming'),
(7, 'Asus TUF Gaming A15', 'Asus', 99000, 86.0, 16, 2024, 15.6, 'Gaming'),
(8, 'Dell Inspiron 15', 'Dell', 62000, 80.0, 8, 2024, 15.6, 'Non-Gaming');
SELECT brand, COUNT(*) FROM Laptops GROUP BY brand;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
SELECT gaming_flag, AVG(spec_score) AS avg_spec_score

```

```

FROM Laptops
GROUP BY gaming_flag;
SELECT ram_gb,
       AVG(price) AS avg_price,
       AVG(spec_score) AS avg_spec_score
FROM Laptops
GROUP BY ram_gb
ORDER BY ram_gb;
SELECT release_year, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY release_year
ORDER BY release_year;
INSERT INTO Laptops
(laptop_id, name, brand, price, spec_score, ram_gb, release_year, screen_inch, gaming_flag)
VALUES
(9, 'HP 15-fd0186TU', 'HP', 52000, 76.0, 8, 2023, 15.6, 'Non-Gaming'),
(10, 'Lenovo IdeaPad 3 14', 'Lenovo', 45000, 77.0, 8, 2022, 14.0, 'Non-Gaming'),
(11, 'Asus Vivobook 15 2021', 'Asus', 43000, 75.0, 8, 2021, 15.6, 'Non-Gaming');
SELECT release_year, COUNT(*) AS laptop_count
FROM Laptops
GROUP BY release_year
ORDER BY release_year;
USE laptop_dss;

```

```

CREATE TABLE Brand (
    brand_id INT PRIMARY KEY,
    brand_name VARCHAR(50)
);

```

```

INSERT INTO Brand (brand_id, brand_name) VALUES
(1, 'HP'),
(2, 'Lenovo'),
(3, 'Asus'),
(4, 'Dell');

```

```

ALTER TABLE Laptops
ADD COLUMN brand_id INT,
ADD CONSTRAINT fk_laptops_brand
FOREIGN KEY (brand_id) REFERENCES Brand(brand_id);
USE laptop_dss;

```

```

ALTER TABLE Laptops
ADD COLUMN brand_id INT;

```

```

ALTER TABLE Laptops
ADD CONSTRAINT fk_laptops_brand
FOREIGN KEY (brand_id) REFERENCES Brand(brand_id);
UPDATE Laptops SET brand_id = 1 WHERE brand = 'HP';
UPDATE Laptops SET brand_id = 2 WHERE brand = 'Lenovo';
UPDATE Laptops SET brand_id = 3 WHERE brand = 'Asus';

```

```
UPDATE Laptops SET brand_id = 4 WHERE brand = 'Dell';
USE laptop_dss;
```

```
ALTER TABLE Laptops
ADD CONSTRAINT fk_laptops_brand
FOREIGN KEY (brand_id) REFERENCES Brand(brand_id);
USE laptop_dss;
DESCRIBE Laptops;
SELECT * FROM Laptops;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
DESCRIBE Laptops;
SELECT brand, AVG(price) AS avg_price
FROM Laptops
GROUP BY brand;
USE laptop_dss;
USE laptop_dss;
```

```
SHOW CREATE TABLE Laptops;
USE laptop_dss;
```

```
ALTER TABLE Laptops
MODIFY laptop_id INT NOT NULL AUTO_INCREMENT;
SHOW CREATE TABLE Laptops;
INSERT INTO Laptops (name, brand_id)
VALUES ('Some model', 1);
INSERT INTO Laptops (name, brand_id)
VALUES ('Some model', 1);
SELECT laptop_id, name, brand_id
FROM Laptops
ORDER BY laptop_id;
INSERT INTO Laptops (name, brand_id)
VALUES
    ('Gaming model', 1),
    ('Office model', 2);
SELECT laptop_id, name, brand_id
FROM Laptops
ORDER BY laptop_id;
USE laptop_dss;
```

```
-- All laptops with their brand names
SELECT l.laptop_id, l.name, b.brand_name
FROM laptops l
JOIN brand b ON l.brand_id = b.brand_id
ORDER BY l.laptop_id;
-- Count how many laptops per brand
SELECT b.brand_name, COUNT(*) AS num_laptops
FROM laptops l
JOIN brand b ON l.brand_id = b.brand_id
```

```
GROUP BY b.brand_name  
ORDER BY num_laptops DESC;  
USE laptop_dss;
```